



C.P.R. Liceo "La Paz"

Proyecto Fin de Ciclo

Desarrollo de Aplicaciones Web

Autor: Mateo Castro Souto Tutor: Jesús Ángel Perez Roca

1. Resumen

Nexly es una red social full-stack desarrollada como proyecto final del ciclo DAW. La aplicación replica las funcionalidades nucleares de plataformas como Twitter/X y Reddit, ofreciendo a los usuarios un espacio para publicar contenido, interactuar mediante reacciones y comentarios anidados, organizarse en comunidades temáticas y comunicarse en tiempo real a través de mensajería privada y notificaciones.

El proyecto se compone de dos aplicaciones independientes.

- El backend está construido con Spring Boot 3.4 sobre Java 21, utiliza MySQL como base de datos relacional y sigue una arquitectura por capas (controllers → services → repositories) con DTOs y mapeo mediante MapStruct. La autenticación es dual: credenciales propias con contraseña cifrada (BCrypt) y OAuth2 con Google, ambas emitiendo JWT para la gestión de sesiones. La seguridad se complementa con verificación de email obligatoria mediante tokens y envío de correos vía SMTP (Mailpit en desarrollo). Las imágenes se almacenan en Cloudinary y la mensajería en tiempo real, junto con las notificaciones, se implementa con WebSocket STOMP sobre SockJS.
- El frontend está desarrollado en React 18 + TypeScript con Vite, gestionando el estado global mediante Zustand y consumiendo la API REST con interceptores que inyectan el JWT en cada petición. La interfaz, totalmente responsive, incluye feed paginado, creación de publicaciones con citación, selector de emojis y GIFs, sistema de flairs, perfiles editables con foto y portada, búsqueda global de usuarios y comunidades, chat con envío optimista y reintento automático, y un sistema de notificaciones en tiempo real con badge de no leídos.

El proyecto incorpora además un panel de administración accesible solo para usuarios con rol ADMIN, que permite consultar estadísticas del sistema y gestionar usuarios, publicaciones y comunidades. La carga inicial de datos se realiza mediante un script data.sql idempotente que crea 10 usuarios de prueba, 5 comunidades y diversas publicaciones para facilitar la evaluación.

2. Abstract

Nexly is a full-stack social network developed as the final project for the DAW (Web Application Development) higher degree. The application replicates the core functionalities of platforms like Twitter/X and Reddit, providing users with a space to publish content, interact through reactions and nested comments, organize into thematic communities, and communicate in real time via private messaging and notifications.

The project consists of two independent applications:

- Backend: Built with Spring Boot 3.4 on Java 21, it uses MySQL as its relational database and follows a layered architecture (controllers → services → repositories) utilizing DTOs and mapping via MapStruct. It features a dual authentication system: native credentials with encrypted passwords (BCrypt) and OAuth2 with Google, both issuing JWTs for session management. Security is reinforced with mandatory email verification via tokens and SMTP email delivery (Mailpit in development). Images are stored in Cloudinary, while real-time messaging and notifications are implemented using WebSocket STOMP over SockJS.
- Frontend: Developed in React 18 + TypeScript with Vite, managing global state through Zustand and consuming the REST API via interceptors that inject the JWT into every request. The fully responsive interface includes a paginated feed, post creation with quoting, an emoji and GIF selector, a flair system, editable profiles with profile and cover photos, global search for users and communities, a chat system with optimistic updates and automatic retries, and a real-time notification system with an unread badge.

Additionally, the project features an Admin Panel accessible exclusively to users with the ADMIN role, allowing them to view system statistics and manage users, posts, and communities. The initial data seed is handled by an idempotent data.sql script that creates 10 test users, 5 communities, and various posts to streamline evaluation.

3. Palabras Clave

Backend — Tecnologías

- **Java 21** — Lenguaje de programación orientado a objetos sobre el que se construye la API. Se utilizan características modernas como records, var y expresiones lambda.
- **Spring Boot 3.4** — Framework que simplifica el desarrollo de aplicaciones Java mediante autoconfiguración, servidor embebido (Tomcat) y un ecosistema de starters.
- **Spring Security** — Módulo que gestiona la autenticación y autorización, configura los filtros JWT y protege los endpoints mediante anotaciones como @PreAuthorize.
- **Spring Data JPA** — Abstracción sobre Hibernate para mapear entidades a tablas de base de datos y generar consultas automáticamente desde interfaces Repository.
- **Hibernate** — Implementación de JPA que actúa como ORM (Object-Relational Mapping) traduciendo objetos Java a registros SQL.
- **MySQL** — Sistema gestor de bases de datos relacional donde se persisten usuarios, publicaciones, comunidades, mensajes y notificaciones.
- **Maven** — Gestor de dependencias y herramienta de construcción del backend, definido en el pom.xml.
- **MapStruct** — Librería que genera código en tiempo de compilación para convertir entre entidades JPA y DTOs, evitando mapeos manuales.
- **Lombok** — Procesador de anotaciones que genera getters, setters, constructores y builders, reduciendo código repetitivo (boilerplate).
- **JWT** (Java JWT) — Librería para firmar, emitir y validar JSON Web Tokens usados en la autenticación stateless.
- **Cloudinary** — Servicio cloud para almacenar y servir imágenes (publicaciones, perfiles, portadas) con transformaciones automáticas.
- **Spring Mail** — Módulo para envío de correos electrónicos vía SMTP, usado en la verificación de email tras el registro.
- **Mailpit** — Servidor SMTP de desarrollo que captura los correos en una bandeja web local sin enviarlos realmente.
- **WebSocket / STOMP / SockJS** — Protocolos de comunicación bidireccional en tiempo real usados para mensajería y notificaciones push.

- **OAuth2** — Protocolo estándar de autorización implementado para permitir el inicio de sesión con cuenta de Google.

Frontend — Tecnologías

- **React 18** — Librería de interfaces de usuario basada en componentes funcionales y hooks.
- **TypeScript** — Superset de JavaScript que añade tipado estático, mejorando la mantenibilidad y reduciendo errores en tiempo de compilación.
- **Vite** — Bundler y servidor de desarrollo de nueva generación, con recarga en caliente (HMR) ultrarrápida.
- **Zustand** — Librería minimalista de gestión de estado global, usada para auth, chat, notificaciones y comunidades.
- **Axios** — Cliente HTTP con interceptores que inyectan automáticamente el JWT en las cabeceras Authorization.
- **React Router** — Librería de enrutamiento del lado cliente para la navegación entre páginas (SPA).
- **CSS con variables**: root — Sistema de theming mediante variables CSS nativas que permite cambios globales de paleta y tipografía.

Conceptos e ideas clave

- **API REST** — Estilo arquitectónico de la API basado en recursos, verbos HTTP (GET, POST, PUT, DELETE) y representaciones JSON.
- **JWT** (JSON Web Token) — Token firmado que contiene la identidad del usuario y se envía en cada petición, permitiendo autenticación stateless.
- **Arquitectura por capas** — Separación estricta en Controller → Service → Repository → Entity, con DTOs en la frontera para no exponer entidades.
- **DTO** (Data Transfer Object) — Objetos planos que viajan entre frontend y backend, desacoplando la API del modelo de dominio.
- **SPA** (Single Page Application) — Aplicación de una sola página HTML donde la navegación ocurre sin recargas, gestionada por React Router.
- **WebSocket bidireccional** — Canal persistente cliente-servidor que permite recibir mensajes y notificaciones sin polling.
- **Mensajería en tiempo real** — Chat entre usuarios con persistencia REST y

entrega instantánea vía STOMP (/user/queue/mensajes).

- **Notificaciones push** — Avisos en tiempo real (nuevo seguidor, reacción, comentario, mensaje) entregados por WebSocket con badge de no leídos.
- **Envío optimista (optimistic UI)** — Estrategia en el chat: el mensaje aparece en la interfaz antes de confirmarse en el servidor, con reintento si falla.
- **Comunidades con roles** — Grupos temáticos con jerarquía ADMIN / MODERADOR / MIEMBRO, solicitudes de ingreso y baneos.
- **Reacciones y comentarios anidados** — Sistema de interacción con publicaciones que admite múltiples niveles de respuesta.
- **Sistema de seguidores y bloqueos** — Grafo de relaciones entre usuarios que condiciona el feed y la mensajería.
- **Verificación de email** — Flujo obligatorio tras el registro: token enviado por correo, bloqueo de login hasta confirmar.
- **Panel de administración** — Vista privilegiada para gestión global de usuarios, publicaciones y comunidades, protegida por rol.
- **Seeding de datos (data.sql)** — Script idempotente que precarga datos de prueba para facilitar la evaluación.
- **Responsive design** — Diseño adaptable a móvil, tablet y escritorio mediante media query y unidades relativas.
- **Cifrado de contraseñas (BCrypt)** — Algoritmo de hashing con salt usado para almacenar contraseñas de forma segura.
- **CORS (Cross-Origin Resource Sharing)** — Política configurada para permitir que el frontend (puerto distinto) consuma la API.
- **Inyección de dependencias (IoC)** — Principio de Spring que gestiona el ciclo de vida de los componentes y sus dependencias.
- **Programación reactiva en UI** — Modelo de React donde los cambios de estado disparan re-renderizados automáticos.
- **Git / GitHub** — Sistema de control de versiones y plataforma de alojamiento, con ramas (main, develop, feat/*) y Pull Requests.
- **Docker** — Contenedorización usada para levantar MySQL y Mailpit en entorno local de desarrollo.

Sumario

	1
1. Resumen	2
2. Abstract	3
3. Palabras Clave	5
4. Introducción y Objetivos	10
4.1. Introducción	10
4.2. Objetivos del proyecto	11
4.2.1. Objetivos técnicos:	11
4.2.2. Objetivos formativos:	11
5. Estado del arte	12
5.1. Aplicaciones similares en el mercado	12
5.1.1. Twitter / X	12
5.1.2. Reddit	13
5.1.3. Instagram	15
5.1.4. Discord	15
5.2. Tecnologías utilizadas y alternativas	16
5.2.1. Backend: Spring Boot (Java 21)	16
5.2.1.1. Por qué se eligió:	16
5.2.1.2. Alternativas valoradas:	16
5.2.2. Base de datos: MySQL	17
5.2.2.1. Por qué se eligió:	17
5.2.2.2. Alternativas valoradas:	17
5.2.3. Autenticación: JWT + OAuth2	19
5.2.3.1. Por qué se eligió:	19
5.2.3.2. Alternativas valoradas:	19
5.2.4. Comunicación en tiempo real: WebSocket con STOMP y SockJS	19
5.2.4.1. Por qué se eligió:	19
5.2.4.2. Alternativas valoradas:	20
5.2.5. Almacenamiento de imágenes: Cloudinary	20
5.2.5.1. Por qué se eligió:	20
5.2.5.2. Alternativas valoradas:	20
5.2.6. Frontend: React + TypeScript + Vite	21
5.2.6.1. Por qué se eligió:	21

5.2.6.2. Alternativas valoradas:	21
5.2.7. Gestión de estado: Zustand	22
5.2.7.1. Por qué se eligió:	22
5.2.7.2. Alternativas valoradas:	22
5.2.8. Estilos: CSS con variables :root	22
5.2.8.1. Por qué se eligió:	22
5.2.8.2. Alternativas valoradas:	23
5.2.9. Herramientas de soporte	23
6. Caso de estudio	24
6.1. Sistema de autenticación seguro y flexible	24
6.1.1. Problemática identificada:	24
6.1.2. Solución desarrollada:	24
6.2. Modelo de identidad y perfil del usuario	25
6.2.1. Problemática identificada:	25
6.2.2. Solución desarrollada:	25
6.3. Feed de publicaciones enriquecido	26
6.3.1. Problemática identificada:	26
6.3.2. Solución desarrollada:	26
6.4. Sistema de comentarios estructurado	27
6.4.1. Problemática identificada:	27
6.4.2. Solución desarrollada:	27
6.5. Comunidades temáticas con jerarquía de roles	28
6.5.1. Problemática identificada:	28
6.5.2. Solución desarrollada:	28
6.6. Mensajería privada en tiempo real	29
6.6.1. Problemática identificada:	29
6.6.2. Solución desarrollada:	29
6.7. Sistema de notificaciones push	29
6.7.1. Problemática identificada:	29
6.7.2. Solución desarrollada:	30
6.8. Búsqueda global integrada	30
6.8.1. Problemática identificada:	30
6.8.2. Solución desarrollada:	30
6.9. Panel de administración	31
6.9.1. Problemática identificada:	31
6.9.2. Solución desarrollada:	31
6.10. Arquitectura técnica robusta y mantenible	31
6.10.1. Problemática identificada:	32
6.10.2. Solución desarrollada:	32
6.11. Experiencia de usuario en cualquier dispositivo	32
6.11.1. Problemática identificada:	32
6.11.2. Solución desarrollada:	33
6.12. Facilidad de evaluación y despliegue	33

6.12.1. Problemática identificada:	33
6.12.2. Solución desarrollada:	33
7. Diseño	35
7.1 Diseño e Interfaz de Usuario	35
7.1.1 Proceso de diseño	35
7.1.1.1 Análisis de requisitos de UI.	35
7.1.1.2 Wireframing.	35
7.1.1.3. Mockup de alta fidelidad.	35
7.1.1.4. Revisión de accesibilidad.	35
7.2 Wireframes	36
7.2.1. Feed principal (Home):	36
7.2.2. Perfil de usuario	37
7.2.3. Explorar / Búsqueda	37
7.2.4. Mensajería	38
7.2.5. Inicio de sesión	39
7.2.6. Registro	40
7.3 Mockups	42
7.4 Decisiones de diseño	44
7.4.1 Layout de tres columnas	44
7.4.2 Navegación adaptativa (sidebar vs barra inferior)	44
7.4.3 Sistema de variables CSS	45
7.4.4 Layout de autenticación en dos paneles	45
7.4.5 Onboarding multi-paso con barra de progreso	45
7.4.6 Skeleton loading	46
7.4.7 Fallback de avatar	46
7.4.8 Variantes de PostCard	46
7.5 Usabilidad	46
7.5.1. Visibilidad del estado del sistema	47
7.5.2. Coincidencia con el mundo real	47
7.5.3. Control del usuario y libertad	47
7.5.4. Consistencia y estándares	47
7.5.5. Prevención de errores	47
7.5.6. Reconocimiento antes que recuerdo	48
7.5.7. Flexibilidad y eficiencia	48
7.5.8. Diseño estético y minimalista	48
7.5.9. Ayuda al usuario a reconocer y recuperarse de errores	48
7.5.10. Ayuda y documentación	48
	48
	48
	49
	49
	49
8. Diagramas	49
8.1. E-R de la(s) base(s) de datos	49
8.2. Diagrama de Casos de Uso	50

8.2.1. Módulo de autenticación.	50
8.2.2. Módulo de Feed y Publicaciones.	50
8.2.3. Módulo de Comunidades	51
8.2.4. Módulo de Chat	52
8.2.5. Módulo de Administrador	53
8.2.6. Explicación de los casos de uso	53
8.3 Diagrama de clases	54
8.3.1. Explicación del diagrama de clases	54
9. Desarrollo del proyecto	56
9.1 Arquitectura general	56
9.2 Estructura del Backend	56
9.3 Desarrollo por módulos	57
9.3.1 Autenticación	57
9.3.2 Feed y publicaciones	59
9.3.3 Comunidades	60
9.3.4 Chat en tiempo real	62
9.3.5 Notificaciones en tiempo real	64
9.4 API REST — principales endpoints	65
10. Forma jurídica y trámites de constitución	66
10.1. Principales trámites de constitución	66
10.1.1 Certificación negativa de denominación social	67
10.1.2 Apertura de cuenta bancaria y desembolso del capital	67
10.1.3. Elaboración de los Estatutos Sociales	67
10.1.4. Escritura pública ante notario	67
10.1.5. Obtención del NIF provisional	67
10.1.6. Liquidación del Impuesto de Transmisiones Patrimoniales (ITP) — modalidad Operaciones Societarias	67
10.1.7. Inscripción en el Registro Mercantil Provincial	67
10.1.8. Alta en la Agencia Tributaria	68
10.1.9 Alta en la Seguridad Social — RETA	68
10.1.10. Licencias y obligaciones sectoriales	68
11. Viabilidad tecno-económica	69
11.1. Costes de desarrollo (inversión inicial)	69
11.2. Costes de infraestructura y operación	69
11.2.1 Escenario A — Lanzamiento / hasta ~500 usuarios activos	69
11.2.2. Escenario B — Crecimiento / 500–10.000 usuarios activos	70
11.3. Modelo de monetización y análisis de rentabilidad	70
11.3.1 Punto de equilibrio (break-even)	70
11.4. Tabla resumen — Coste total primer año	71
11.5. Conclusión de viabilidad	71
12. Trabajo futuro	72
12.1 Nuevas funcionalidades	72
12.2. Mejoras técnicas y de rendimiento	73
12.3 Mejoras de experiencia de usuario	73

13. Conclusiones	75
13.1 Objetivos alcanzados	75
13.2. Objetivos no alcanzados y causas	75
13.3. Valoración personal	76
14. Biblioteca de recursos web y referencias	77

4. Introducción y Objetivos

4.1. Introducción

La elección de desarrollar una red social como proyecto de fin de ciclo no fue casual. Tras valorar diferentes ideas — una tienda online, un gestor de tareas, una plataforma de reservas — se optó por Nexly porque representa el tipo de aplicación más completo y exigente que un desarrollador full-stack puede afrontar en su formación inicial: un sistema con múltiples actores que interactúan entre sí en tiempo real, donde casi cada decisión técnica tiene implicaciones de seguridad, rendimiento y experiencia de usuario.

Una red social, a diferencia de un CRUD tradicional, obliga a resolver problemas que no se enseñan de forma directa en el ciclo y que solo aparecen cuando se construye algo cercano a un producto real: autenticación robusta con tokens y proveedores externos, comunicación bidireccional mediante WebSockets, gestión de archivos en la nube, relaciones complejas entre entidades (seguidores, bloqueos, roles dentro de comunidades), notificaciones push, paginación eficiente, moderación, verificación de identidad y un largo etcétera. Afrontar este reto suponía salir de la zona de confort del CRUD básico para entrar en el terreno de las aplicaciones que la gente realmente usa cada día.

A ello se suma una motivación personal: las redes sociales son, hoy, uno de los productos software de mayor impacto cultural y técnico del mundo. Entender cómo funcionan por dentro — cómo se entrega un mensaje al instante, cómo se cargan miles de publicaciones sin colapsar el servidor, cómo se protege la cuenta de un usuario — resulta tan atractivo como formativo. Construir una desde cero, aunque sea a escala reducida, permite enfrentarse a los mismos problemas que resuelven los equipos de ingeniería de las plataformas comerciales, y aprender en el proceso muchísimo más que replicando un ejemplo guiado.

Por último, una red social es un proyecto que se puede enseñar y demostrar fácilmente. Es visual, interactivo y comprensible para cualquier evaluador, técnico o no: basta con abrir la aplicación, registrar dos cuentas y verlas hablar en tiempo real para apreciar el alcance del trabajo. Esto la convertía en una

candidata ideal para la defensa final del ciclo.

4.2. Objetivos del proyecto

El desarrollo de Nexly persigue una serie de objetivos claramente definidos, tanto técnicos como formativos:

4.2.1. Objetivos técnicos:

- Dominar un stack profesional moderno y demandado en el mercado laboral, combinando un backend en Spring Boot con Java 21 y un frontend en React con TypeScript, dos de las tecnologías más utilizadas hoy en el desarrollo full-stack.
- Diseñar e implementar una API REST completa, aplicando una arquitectura por capas limpia (Controller → Service → Repository) con uso disciplinado de DTOs y mapeo automático.
- Modelar una base de datos relacional compleja que represente fielmente las relaciones entre usuarios, publicaciones, comentarios, reacciones, comunidades y mensajes, manteniendo la integridad referencial y el rendimiento.
- Implementar un sistema de autenticación seguro y dual, soportando tanto credenciales propias (con cifrado BCrypt y verificación por email) como inicio de sesión federado con Google mediante OAuth2.
- Incorporar comunicación en tiempo real a través de WebSockets con STOMP, indispensable tanto para la mensajería privada como para las notificaciones push.
- Integrar servicios externos en la nube, en particular Cloudinary para el almacenamiento de imágenes, aprendiendo a delegar responsabilidades fuera del propio servidor.
- Construir una interfaz de usuario moderna, responsive y accesible, capaz de funcionar con fluidez tanto en móvil como en escritorio.

4.2.2. Objetivos formativos:

- Afrontar el desarrollo de un proyecto completo de principio a fin, asumiendo todos los roles: análisis, diseño, programación, pruebas, despliegue y documentación.
- Adquirir experiencia real con Git y GitHub como herramientas de trabajo profesional, usando ramas, Pull Requests y revisiones de código.
- Aprender a tomar decisiones técnicas razonadas, evaluando alternativas (REST vs WebSocket, JWT vs sesiones, almacenamiento local vs cloud) y justificando la elección.
- Desarrollar la capacidad de leer documentación oficial y resolver problemas de forma autónoma, una habilidad clave en cualquier puesto de desarrollador.

5. Estado del arte

5.1. Aplicaciones similares en el mercado

El terreno de las redes sociales es uno de los más maduros y competitivos del software actual. Nexly se inspira, en mayor o menor medida, en varias plataformas consolidadas. A continuación se analizan las más representativas, comparando sus puntos fuertes y débiles con el enfoque adoptado en este proyecto.

5.1.1. Twitter / X

Puntos fuertes:

- Modelo de publicación corto, directo y viral, con un feed cronológico/algorítmico extremadamente optimizado.
- Sistema de citación de publicaciones (quote tweets) que enriquece el debate público.
- Infraestructura de tiempo real masiva: notificaciones, trending topics y mensajería instantánea a escala global.
- API pública (hoy de pago) que ha dado pie a miles de integraciones de

terceros.

Puntos débiles:

- Moderación históricamente irregular: problemas serios de acoso, bots y desinformación.
- Falta de espacios temáticos delimitados: todo el mundo habla en la misma plaza pública.
- Cambios bruscos de producto desde 2022 (rebranding, cierre de API gratuita, monetización agresiva) que han alienado aparte de la comunidad.

Comparativa con Nexly: Nexly retoma la idea del feed con publicaciones cortas, reacciones y citación, pero la combina con comunidades temáticas con roles — algo que Twitter nunca ha terminado de implementar bien — ofreciendo así un híbrido entre plaza pública y foro estructurado.

5.1.2. Reddit

Puntos fuertes:

- Estructura en subreddits que organiza el contenido por intereses, con moderadores propios y reglas internas.
- Sistema de votación (upvote/downvote) que jerarquiza el contenido por relevancia comunitaria.
- Hilos de comentarios profundamente anidados, ideales para debates largos.
- Cultura de pseudonimato que favorece la libertad de expresión.

Puntos débiles:

- Interfaz históricamente confusa para usuarios nuevos.
- Mensajería privada minimalista y poco usada.
- Experiencia móvil que durante años fue muy inferior a la web.

Comparativa con Nexly: Nexly toma de Reddit el concepto de comunidades con roles (ADMIN, MODERADOR, MIEMBRO), solicitudes de ingreso, baneos y comentarios anidados, pero los integra con un chat en tiempo real de primera clase y una interfaz moderna y responsive, dos áreas en las que Reddit ha mostrado debilidades históricas.

5.1.3. Instagram

Puntos fuertes:

- Diseño visual cuidadísimo y centrado en la imagen.
- Perfiles muy personalizables, con foto, biografía y portada.
- Mensajería integrada con stickers, reacciones y llamadas.

Puntos débiles:

- Algoritmo de feed opaco y muy comercial.
- Falta de funcionalidades centradas en el texto y el debate.
- Dependencia total de la app móvil: la versión web es funcional pero secundaria.

Comparativa con Nexly: De Instagram, Nexly toma la importancia de la identidad visual del perfil (foto, portada, biografía editables) y el uso de imágenes en publicaciones, pero con un enfoque que da el mismo peso al texto y al debate, no solo a la estética.

5.1.4. Discord

Puntos fuertes:

- Comunicación en tiempo real excepcional, con canales por tema y servidores temáticos.
- Mensajería bidireccional muy fluida basada en WebSockets.
- Roles, permisos granulares y moderación avanzada.

Puntos débiles:

- Curva de aprendizaje alta para usuarios no técnicos.
- Ausencia de un feed público al estilo red social: todo es conversación.
- Indexabilidad casi nula: el contenido vive cerrado dentro de cada servidor.

Comparativa con Nexly: Nexly comparte con Discord el uso de WebSocket con STOMP para mensajería y notificaciones en tiempo real, pero combina ese canal en vivo con un feed público y persistente indexable mediante búsqueda — el equilibrio justo entre conversación y publicación que Discord no busca.

5.2. Tecnologías utilizadas y alternativas

A continuación se justifican las principales decisiones técnicas del proyecto, contrastándolas con las alternativas reales que se valoraron.

5.2.1. Backend: Spring Boot (Java 21)

5.2.1.1. Por qué se eligió:

- Estándar de facto en el desarrollo de aplicaciones empresariales en Europa, con altísima demanda laboral.
- Ecosistema maduro: Spring Security, Spring Data, Spring WebSocket, Spring Mail, OAuth2 client, todo integrado y autoconfigurado.
- Java 21 (LTS) aporta records, pattern matching y mejoras de rendimiento.
- Comunidad enorme y documentación oficial impecable.

5.2.1.2. Alternativas valoradas:

- Node.js + Express/NestJS – Más ligero y con tiempos de arranque menores; el mismo lenguaje en front y back. Descartado por preferir aprender una tecnología fuertemente tipada y orientada a objetos.
- Django (Python) – Productividad muy alta con su ORM y panel admin gratuito. Descartado por menor presencia en el mercado local de empleo.
- ASP.NET Core (C#) – Excelente rendimiento, similar a Spring Boot en filosofía. Descartado por menor familiaridad previa con el ecosistema .NET.

5.2.2. Base de datos: MySQL

5.2.2.1. Por qué se eligió:

- Base de datos relacional más extendida del mundo, gratuita y multiplataforma.
- Integración perfecta con Spring Data JPA / Hibernate.
- Soporte completo de transacciones, claves foráneas e índices, imprescindibles para un modelo con muchas relaciones.

5.2.2.2. Alternativas valoradas:

- PostgreSQL – Técnicamente superior en muchos aspectos (tipos JSON nativos, índices avanzados, extensiones). Habría sido una elección

igualmente válida.

- MongoDB — Base de datos documental NoSQL. Descartada porque el modelo de Nexly es marcadamente relacional (usuarios, comunidades, mensajes, roles) y forzar un modelo documental complicaría más de lo que simplificaría.
- SQLite — Útil para prototipos, insuficiente para una app multiusuario en producción.

5.2.3. Autenticación: JWT + OAuth2

5.2.3.1. Por qué se eligió:

- JWT permite una autenticación stateless, ideal para arquitecturas escalables: el servidor no guarda sesiones en memoria.
- OAuth2 con Google ofrece al usuario una vía de registro instantáneo y segura, sin gestionar otra contraseña.

5.2.3.2. Alternativas valoradas:

- Sesiones tradicionales con cookies — Más simples de invalidar, pero requieren almacenamiento en servidor y complican el escalado horizontal.
- Auth0 / Firebase Auth / Clerk — Soluciones gestionadas que aceleran enormemente el desarrollo. Descartadas porque uno de los objetivos formativos era implementar la autenticación desde cero para aprender cómo funciona internamente.

5.2.4. Comunicación en tiempo real: WebSocket con STOMP y SockJS

5.2.4.1. Por qué se eligió:

- WebSocket es el estándar para comunicación bidireccional persistente entre cliente y servidor.
- STOMP aporta un protocolo de mensajería con destinos (/user/queue/...) y suscripciones, perfectamente integrado en Spring.
- SockJS garantiza compatibilidad con entornos donde WebSocket esté bloqueado, cayendo a long-polling automáticamente.

5.2.4.2. Alternativas valoradas:

- Polling / long-polling puro — Más sencillo, pero ineficiente: el cliente pregunta una y otra vez si hay novedades.
- Server-Sent Events (SSE) — Útil para flujos unidireccionales (servidor → cliente). Insuficiente para chat, donde se necesita comunicación bidireccional.
- Socket.IO (Node.js) — Excelente librería, pero requiere backend en Node, incompatible con la elección de Spring Boot.

5.2.5. Almacenamiento de imágenes: Cloudinary

5.2.5.1. Por qué se eligió:

- Sube, almacena, transforma y sirve imágenes optimizadas con CDN global incluido.
- Tier gratuito generoso, suficiente para un proyecto académico y demos.
- API muy sencilla desde Java.

5.2.5.2. Alternativas valoradas:

- Almacenamiento local en el servidor — Sencillo, pero no escalable: ocupa disco, no se replica y no aprovecha CDN.
- Amazon S3 — El estándar profesional. Más complejo de configurar y con coste asociado desde el primer momento.
- Firebase Storage — Buena alternativa, pero menos integrada con Java/Spring que Cloudinary.

5.2.6. Frontend: React + TypeScript + Vite

5.2.6.1. Por qué se eligió:

- React es la librería de UI más demandada del mercado y con la mayor comunidad.
- TypeScript añade tipado estático, reduciendo errores en tiempo de desarrollo y mejorando la mantenibilidad.
- Vite ofrece un servidor de desarrollo extremadamente rápido y un build

optimizado, muy superior a Create React App.

5.2.6.2. Alternativas valoradas:

- Angular — Framework completo con todo incluido. Descartado por curva de aprendizaje más pronunciada y mayor verbosidad.
- Vue — Excelente alternativa, más sencilla de iniciar. Descartada por menor demanda laboral relativa frente a React.
- Next.js — Framework sobre React con SSR y SSG integrados. Descartado por no necesitar renderizado en servidor para este caso de uso.

5.2.7. Gestión de estado: Zustand

5.2.7.1. Por qué se eligió:

- Mínima API, sin boilerplate.
- Stores independientes por dominio (auth, chat, notificaciones, comunidades).
- Excelente integración con TypeScript.

5.2.7.2. Alternativas valoradas:

- Redux / Redux Toolkit — Estándar histórico, pero verboso. Sobredimensionado para este proyecto.

- Context API de React — Suficiente para casos simples; problemas de rendimiento con actualizaciones frecuentes (chat en vivo).
- Recoil / Jotai — Buenas alternativas atómicas; Zustand se eligió por mayor madurez y popularidad.

5.2.8. Estilos: CSS con variables :root

5.2.8.1. Por qué se eligió:

- Variables CSS nativas, soportadas por todos los navegadores modernos.
- Permiten un sistema de theming centralizado y fácilmente mantenible.
- Sin dependencias externas ni proceso de compilación adicional.

5.2.8.2. Alternativas valoradas:

- Tailwind CSS — Muy productivo, pero genera markup verboso y opaco.
- CSS-in-JS (styled-components, Emotion) — Buena DX, pero coste de runtime y bundle.
- Sass / Less — Preprocesadores clásicos; las variables CSS nativas han hecho gran parte de su uso redundante.

5.2.9. Herramientas de soporte

- Maven — Gestor de dependencias del backend, estándar en proyectos

Java empresariales. Alternativa: Gradle, más flexible pero con curva de aprendizaje mayor.

- MapStruct — Generación automática de mappers entidad↔DTO en tiempo de compilación. Alternativa: ModelMapper (basado en reflexión, menos eficiente).
- Lombok — Reduce el boilerplate de Java. Alternativa: usar records para DTOs y aceptar más código manual.
- Mailpit — Servidor SMTP local para desarrollo. Alternativas: MailHog (similar, menos mantenido) o un SMTP real con cuenta de pruebas.
- Docker — Contenedorización de MySQL y Mailpit para entorno local. Alternativa: instalar los servicios directamente en la máquina, con peor portabilidad.
- Git + GitHub — Control de versiones y alojamiento. Alternativa: GitLab o Bitbucket, igualmente válidos.

6. Caso de estudio

Una vez identificadas las limitaciones de las plataformas existentes en el Estado del arte, el desarrollo de Nexly se plantea como una propuesta integradora: una aplicación que aborde, en una sola plataforma coherente, las funcionalidades que en el mercado actual aparecen dispersas o resueltas de forma deficiente. A continuación se describen, ordenados por dominio funcional, los puntos concretos que el proyecto desarrolla para responder a esa problemática.

6.1. Sistema de autenticación seguro y flexible

6.1.1. Problemática identificada:

Muchas plataformas obligan al usuario a registrarse con credenciales propias, generando fricción en el alta. Otras delegan toda la autenticación en proveedores externos, dejando sin alternativa a quien no quiere usarlos. Además, abundan los registros sin verificación que permiten el alta masiva de cuentas falsas.

6.1.2. Solución desarrollada:

- Doble vía de registro: credenciales propias (email + contraseña) y OAuth2 con Google, ofreciendo al usuario elegir.
- Cifrado de contraseñas con BCrypt, algoritmo con salt aleatorio resistente a ataques de fuerza bruta y rainbow tables.
- Verificación obligatoria de email mediante token de un solo uso enviado al correo del usuario. El login queda bloqueado con respuesta 403 EMAIL_NO_VERIFICADO hasta que se confirme.
- Autenticación stateless con JWT: el servidor no mantiene sesiones en memoria, lo que facilita el escalado horizontal y reduce el consumo de recursos.
- Marcado automático como verificado en el registro vía Google, ya que el proveedor garantiza la identidad del email.
- Reenvío de token de verificación disponible desde la pantalla de login para

los casos en que el correo se pierda o caduque.

6.2. Modelo de identidad y perfil del usuario

6.2.1. Problemática identificada:

En redes orientadas al texto (Twitter, Reddit) los perfiles son secundarios y poco expresivos. En las visuales (Instagram) son ricos pero rígidos. Faltan plataformas donde el perfil sea a la vez completo y editable sin fricciones.

6.2.2. Solución desarrollada:

- Perfil editable con nombre, biografía, foto de perfil y foto de portada, todo modificable desde un modal sin salir de la página.
- Onboarding inicial que guía al usuario recién registrado a completar su perfil antes de empezar a publicar.
- Subida de imágenes a Cloudinary con carpetas separadas (perfiles, portadas, publicaciones) y URLs servidas desde CDN global.
- Sistema de seguidores bidireccional (seguir / dejar de seguir) que condiciona el feed.
- Sistema de bloqueos que impide que un usuario bloqueado vea, comente o envíe mensajes al bloqueador.

6.3. Feed de publicaciones enriquecido

6.3.1. Problemática identificada:

Los feeds modernos sufren dos males opuestos: opacidad algorítmica (Instagram, X) o desorganización temática (Twitter sin filtros). Además, la mayoría limita el formato de publicación a texto plano o imagen, ignorando recursos expresivos como GIFs o emojis nativos.

6.3.2. Solución desarrollada:

- Feed paginado con carga progresiva (infinite scroll) para minimizar el consumo de datos y memoria.
- Publicaciones con texto, imagen, emojis y GIFs seleccionables desde pickers integrados.
- Sistema de citación (quote): permite republicar una publicación añadiendo contexto propio, mostrando la original incrustada como QuoteCard.
- Reacciones múltiples a publicaciones (no solo "me gusta"), permitiendo expresar emociones más matizadas.
- Flairs: etiquetas visuales para categorizar publicaciones dentro de comunidades, facilitando el filtrado temático.

6.4. Sistema de comentarios estructurado

6.4.1. Problemática identificada:

El comentario plano (Instagram) limita el debate. El comentario excesivamente anidado (Reddit clásico) lo vuelve ilegible en móvil.

6.4.2. Solución desarrollada:

- Comentarios anidados con respuestas a respuestas, mostrados con indentación clara.
- Reacciones a comentarios, no solo a publicaciones, para valorar aportaciones individuales sin generar ruido textual.
- Carga eficiente mediante store dedicado (CommentSection) que evita re-renderizados innecesarios.

6.5. Comunidades temáticas con jerarquía de roles

6.5.1. Problemática identificada:

Twitter no las tiene, Discord las cierra en servidores aislados sin descubrimiento, Reddit las ofrece pero con interfaz anticuada y mensajería interna pobre.

6.5.2. Solución desarrollada:

- Comunidades abiertas y cerradas: las cerradas requieren solicitud de ingreso aprobada por moderación.
- Tres niveles de rol (ADMIN, MODERADOR, MIEMBRO) con permisos diferenciados sobre publicaciones, miembros y configuración.
- Gestión de solicitudes de ingreso desde un panel dedicado de la

comunidad.

- Sistema de baneos que impide la reincorporación de usuarios expulsados.
- Publicación desde comunidad: las publicaciones quedan asociadas a la comunidad de origen y visibles en su detalle.
- Búsqueda global de comunidades mediante endpoint `/api/comunidades/buscar?q=` con resultados en dropdown.

6.6. Mensajería privada en tiempo real

6.6.1. Problemática identificada:

Pocas redes sociales tratan el chat como ciudadano de primera clase. En Reddit es marginal, en Twitter está separado del feed, en Instagram funciona bien pero solo en móvil.

6.6.2 Solución desarrollada:

- Doble canal: REST para historial y persistencia, WebSocket STOMP sobre SockJS para entrega instantánea en `/user/queue/mensajes`.
- Envío optimista: el mensaje aparece inmediatamente en la interfaz mientras se confirma con el servidor; si falla, se reintenta automáticamente y se marca visualmente.
- Badge de mensajes no leídos en el sidebar, actualizado en tiempo real sin necesidad de refrescar.

- Fallback automático a long-polling vía SockJS para entornos donde WebSocket esté bloqueado (redes corporativas, proxies estrictos).

6.7. Sistema de notificaciones push

6.7.1. Problemática identificada:

Las notificaciones suelen llegar con retraso (polling) o requieren refrescar la página. Esto rompe la sensación de tiempo real propia de una red social.

6.7.2. Solución desarrollada:

- Notificaciones entregadas por WebSocket en /user/queue/notificaciones simultáneamente a la persistencia REST.
- Badge con contador de no leídas en el sidebar, sincronizado en vivo entre pestañas y dispositivos.
- Centro de notificaciones donde el usuario puede consultar el historial completo, marcar como leídas o navegar al recurso origen con un clic.

6.8. Búsqueda global integrada

6.8.1. Problemática identificada:

En muchas plataformas, la búsqueda devuelve solo un tipo de entidad (solo usuarios, o solo publicaciones), obligando a navegar entre secciones distintas.

6.8.2. Solución desarrollada:

- Búsqueda unificada de usuarios y comunidades desde un único campo en la cabecera.
- Resultados en dropdown con previsualización instantánea según se escribe, sin esperar a confirmar.
- Página dedicada BusquedaPage con resultados ampliados cuando el usuario quiere explorar a fondo.

6.9. Panel de administración

6.9.1. Problemática identificada:

En proyectos académicos suele omitirse la capa administrativa, dejando la moderación como tarea manual sobre la base de datos. Esto no refleja cómo funcionan las plataformas reales.

6.9.2. Solución desarrollada:

- Sección /admin protegida con `@PreAuthorize("hasRole('ADMIN')")` en backend y enlace visible en el sidebar solo si `usuario.rol === 'ADMIN'` en frontend.
- Dashboard con estadísticas globales: total de usuarios, publicaciones y comunidades activas.

- Gestión de usuarios: cambio de rol, activación/desactivación de cuenta, borrado lógico.
- Gestión de publicaciones y comunidades: borrado moderado desde la propia interfaz.
- Endpoints diferenciados bajo `/api/admin/**` que devuelven 403 a cualquier usuario sin rol ADMIN.

6.10. Arquitectura técnica robusta y mantenible

6.10.1. Problemática identificada:

Muchos proyectos académicos mezclan responsabilidades (lógica en controllers, exposición de entidades JPA al cliente, queries SQL dispersas) generando código difícil de mantener.

6.10.2. Solución desarrollada:

- Arquitectura por capas estricta: Controller → Service → Repository → Entity, con DTOs en la frontera entre la API y el cliente.
- Mapeo automático con MapStruct generado en tiempo de compilación, sin reflexión y sin código manual repetitivo.
- Reducción de boilerplate con Lombok (`@Data`, `@Builder`, `@RequiredArgsConstructor`).
- Stores de Zustand por dominio en frontend, evitando un estado global

monolítico.

- Interceptores Axios que inyectan el JWT automáticamente en cada petición, centralizando la lógica de autenticación.
- Configuración segura desde `application.properties` con variables de entorno para credenciales sensibles (BD, Cloudinary, OAuth2, SMTP).

6.11. Experiencia de usuario en cualquier dispositivo

6.11.1. Problemática identificada:

Muchas redes sociales fallan en escritorio (Instagram) o en móvil (Reddit clásico). Pocas ofrecen una experiencia equivalente en ambos.

6.11.2. Solución desarrollada:

- Diseño completamente responsive mediante media queries y unidades relativas.
- Sistema de theming centralizado con variables CSS en `:root`, permitiendo cambios globales coherentes.
- Componentes accesibles: navegación por teclado, contrastes adecuados y feedback visual en acciones.

6.12. Facilidad de evaluación y despliegue

6.12.1. Problemática identificada:

Los proyectos académicos son a menudo difíciles de probar por el tribunal: bases de datos vacías, dependencias sin documentar, configuraciones opacas.

6.12.2. Solución desarrollada:

- Script data.sql idempotente que precarga 10 usuarios, 5 comunidades, 16 publicaciones y diversas relaciones, con contraseña común Nexly1234 para facilitar la evaluación.
- Cuenta de administrador preconfigurada (admin@nexly.com) para inspeccionar el panel administrativo sin pasos adicionales.
- docker compose para MySQL y Mailpit, dejando los servicios listos con un único comando.
- README en raíz con instrucciones completas de arranque, variables de entorno requeridas y estructura del proyecto.

7. Diseño

7.1 Diseño e Interfaz de Usuario

7.1.1 Proceso de diseño

El diseño de Nexly siguió un proceso iterativo dividido en cuatro fases secuenciales:

7.1.1.1 Análisis de requisitos de UI.

A partir de los casos de uso definidos se identificaron las pantallas necesarias: autenticación (login y registro), feed principal, comunidades, chat privado, perfil de usuario, búsqueda, notificaciones y panel de administración. Cada pantalla quedó asociada a un actor y a los flujos funcionales del sistema.

7.1.1.2 Wireframing.

Se crearon wireframes de baja fidelidad para las ocho pantallas principales. En esta fase el objetivo era definir la disposición de los bloques de contenido, la jerarquía de la información y la navegación global, sin aplicar color ni estilo visual.

7.1.1.3. Mockup de alta fidelidad.

Sobre la estructura definida en los wireframes se elaboraron los mockups aplicando el sistema de diseño definitivo: paleta de color, tipografía, espaciados y componentes reutilizables.

7.1.1.4. Revisión de accesibilidad.

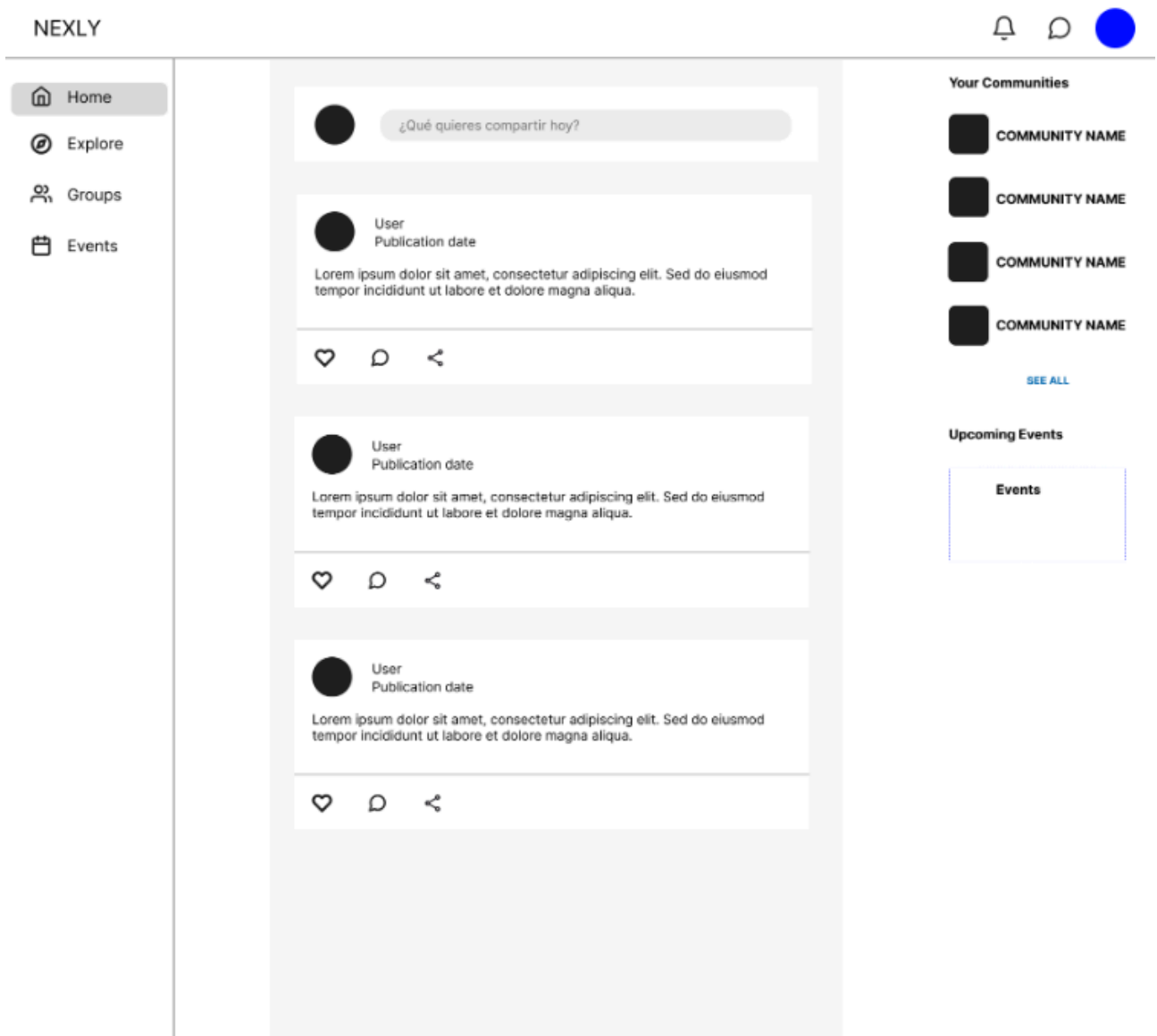
Antes del cierre del diseño se realizó una auditoría de contraste de color y navegación por

teclado cuyos resultados se recogen en el apartado 5.6.

7.2 Wireframes

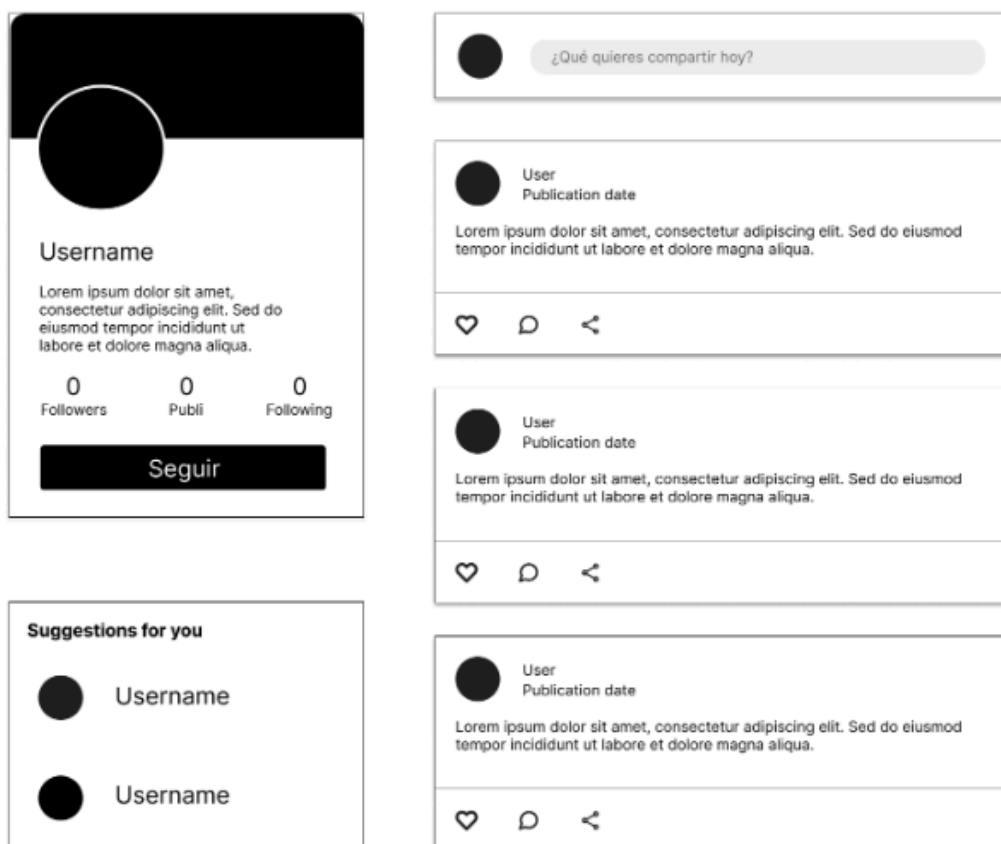
7.2.1. Feed principal (Home):

Pantalla central de la aplicación. La barra lateral izquierda contiene la navegación principal: Home, Explore, Groups y Events. El área central muestra el feed de publicaciones con avatar del autor, fecha, contenido y acciones de reacción, comentario y compartir. El panel derecho muestra las comunidades del usuario y los próximos eventos. El usuario puede crear una nueva publicación desde el campo superior del feed.



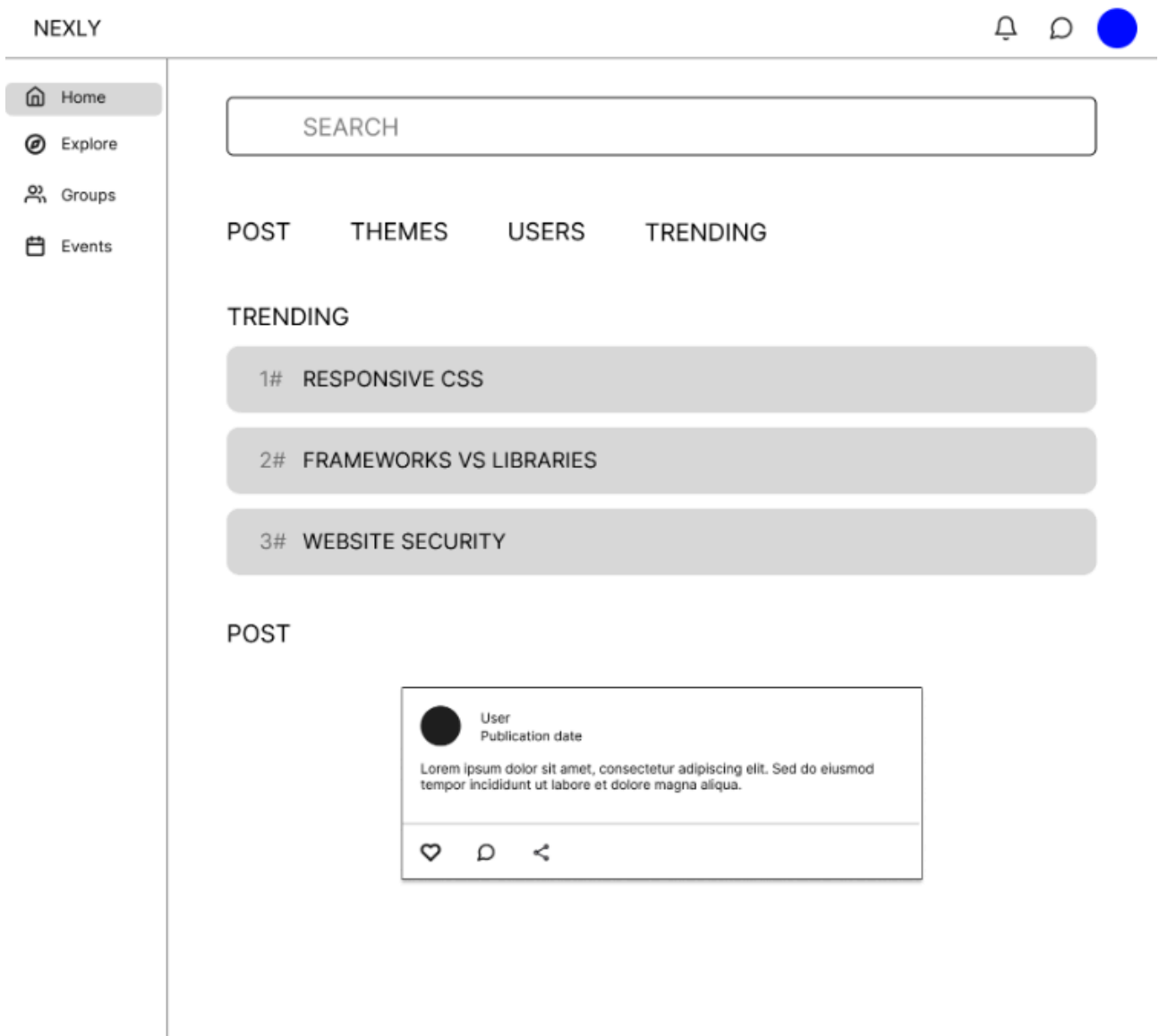
7.2.2. Perfil de usuario

Muestra la foto de portada, avatar, nombre de usuario, biografía y contadores de seguidores, publicaciones y seguidos. Incluye un botón de seguir/dejar de seguir y una sección de sugerencias de usuarios. El feed del perfil muestra únicamente las publicaciones propias del usuario.



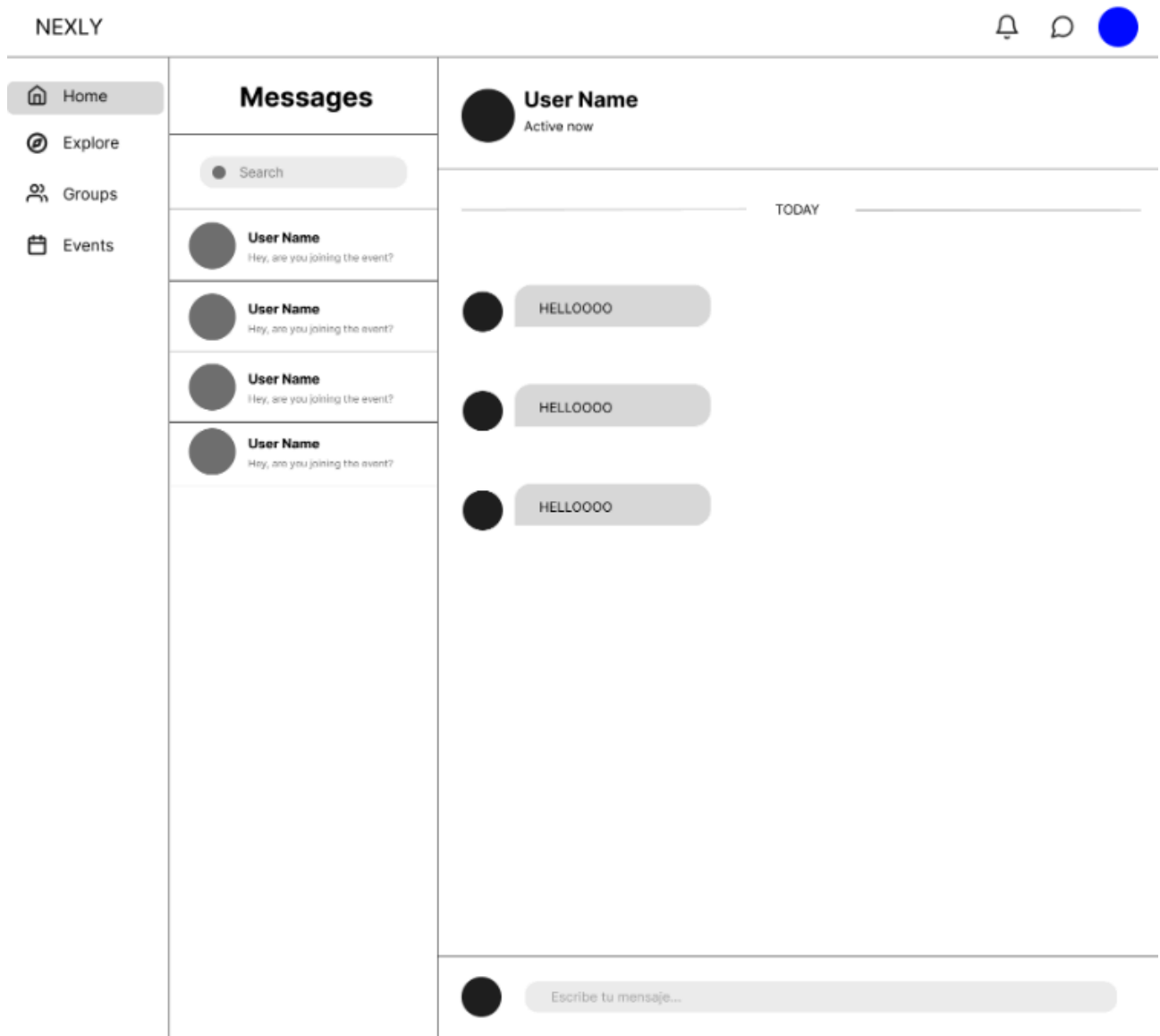
7.2.3. Explorar / Búsqueda

Accesible desde el menú lateral. Contiene una barra de búsqueda en la parte superior y pestañas para filtrar resultados por: Post, Themes, Users y Trending. La sección Trending muestra los temas más populares del momento en forma de ranking. Debajo se muestran publicaciones relacionadas con la búsqueda activa



7.2.4. Mensajería

Panel dividido en dos columnas. La columna izquierda lista las conversaciones activas con buscador integrado. La columna derecha muestra la conversación seleccionada con el historial de mensajes agrupados por fecha y un campo de escritura en la parte inferior.



7.2.5. Inicio de sesión

Formulario de acceso con opción de login social mediante Google y Apple, o bien mediante email y contraseña. Incluye checkbox de "Remember me" y enlace de acceso rápido al registro.

CONNECT WITH YOUR COMMUNITIES

●

●

●

WELCOME BACK

Log in to your Nexly account

CONTINUE WITH

OR WITH EMAIL

EMAIL ADDRESS

Password

☐ Remember me

7.2.6. Registro

Formulario de creación de cuenta con los campos: nombre, apellido, email, contraseña y confirmación de contraseña. Botón principal de Sign Up y enlace a la pantalla de login para usuarios ya registrados.

CREATE ACCOUNT

Name

Last Name

Email Address

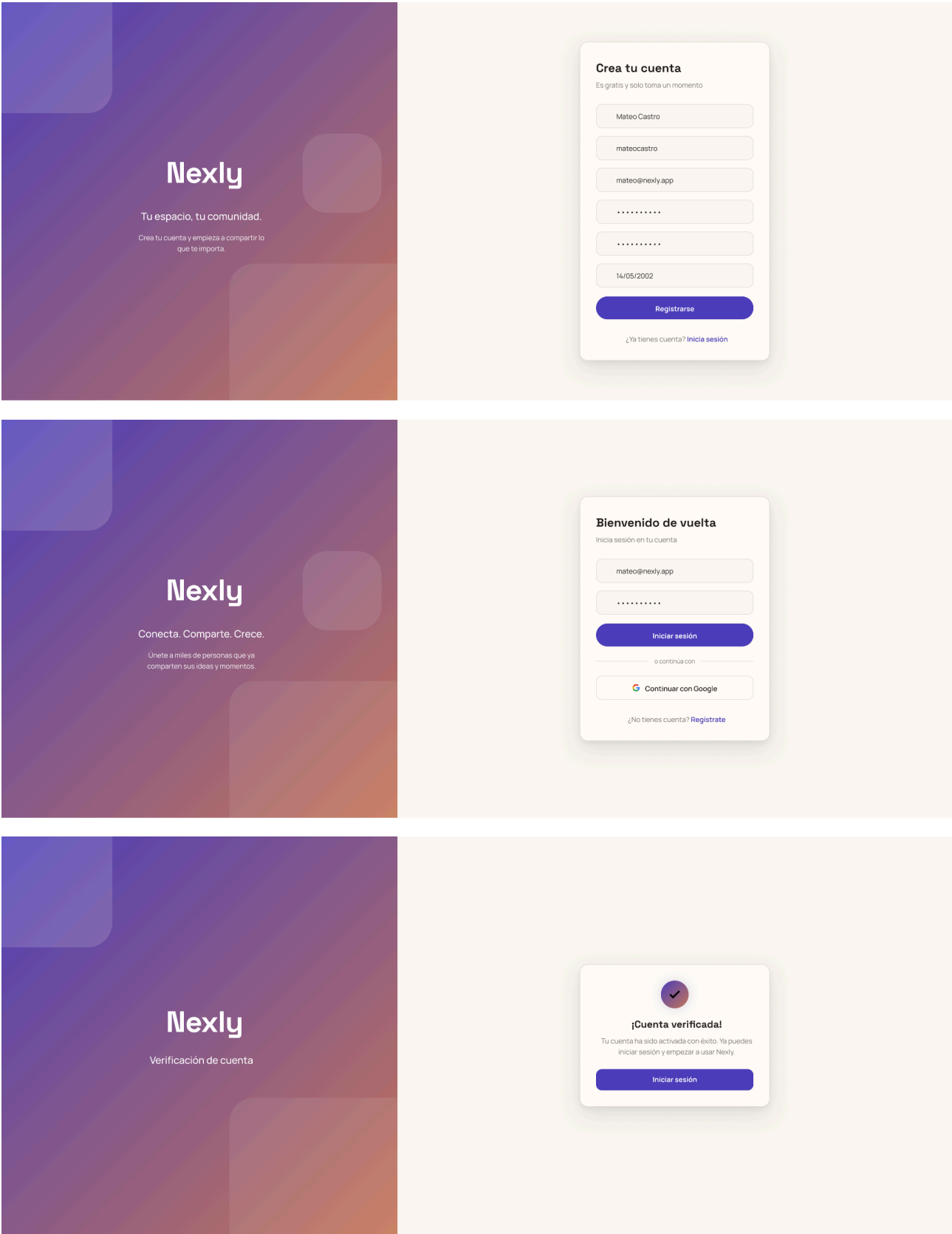
Password

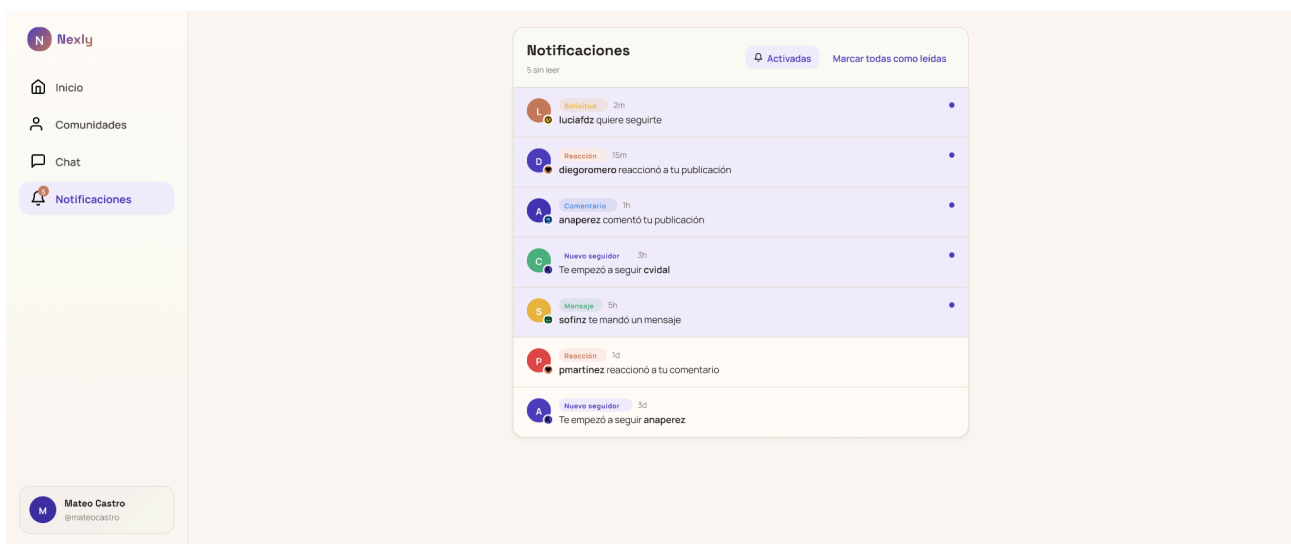
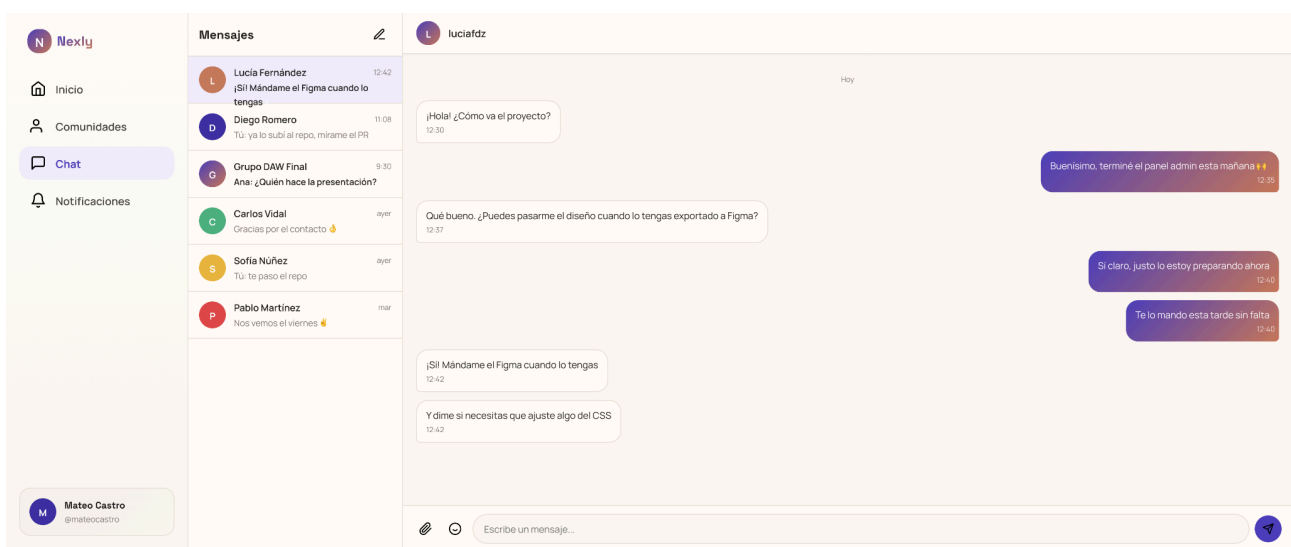
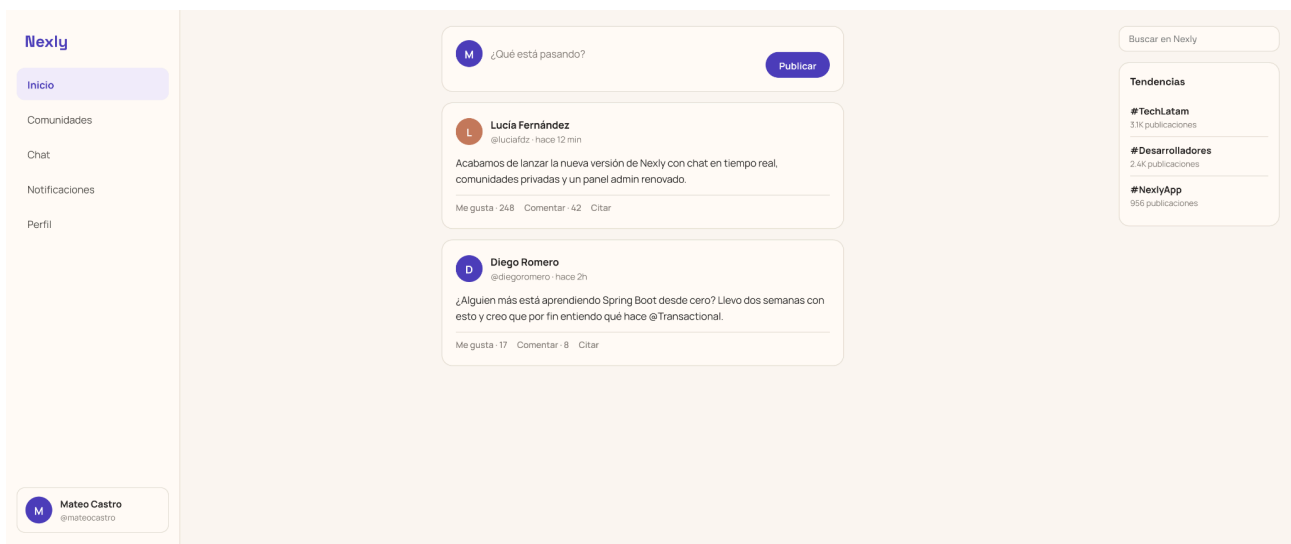
Confirm Password

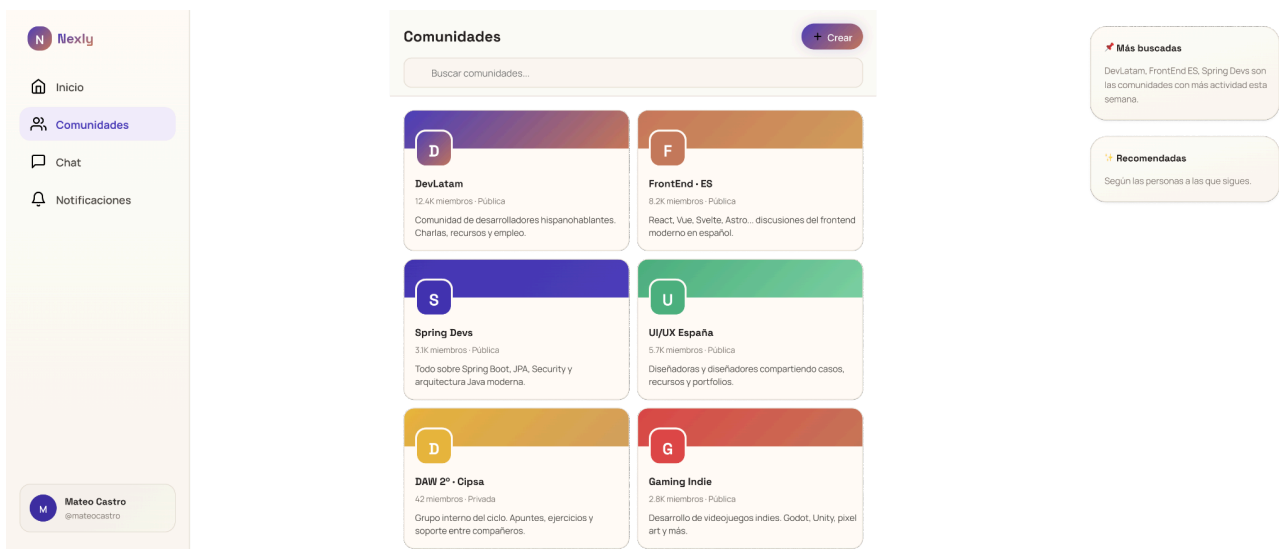
SIGN UP

¿You already have an account? [LOG IN](#)

7.3 Mockups







7.4 Decisiones de diseño

7.4.1 Layout de tres columnas

La aplicación utiliza un layout de tres columnas en pantallas de escritorio: barra lateral de navegación fija a la izquierda, columna central de contenido y panel de descubrimiento a la derecha. Esta disposición sigue el patrón establecido por las principales redes sociales (Twitter/X, Reddit), lo que reduce la curva de aprendizaje del usuario. En la implementación, el panel central está limitado a `max-w-180` para mantener la legibilidad de línea sin importar el ancho del monitor. El panel derecho se suprime por completo cuando el usuario navega al chat (`{!esChat && <DiscoverPanel />}`), eliminando distracciones visuales en una sección que requiere atención sostenida.

7.4.2 Navegación adaptativa (sidebar vs barra inferior)

El componente Sidebar adopta dos formas según el dispositivo. En desktop es una barra lateral fija con logo, etiquetas de texto junto a los iconos y la tarjeta

del usuario activo en la parte inferior. En mobile se convierte en una barra de navegación inferior fija (fixed bottom-0), ocultando el logo y reduciendo las etiquetas a texto de 10px bajo cada icono. Esta estrategia mantiene el acceso a la navegación principal en todo momento sin ocupar espacio vertical en pantallas pequeñas.

7.4.3 Sistema de variables CSS

Toda la paleta de color, gradientes, sombras y tipografía se gestionan mediante custom properties CSS (--color-surface, --color-brand, --gradient-brand, --shadow-lg, --font-display, etc.). Este enfoque centraliza el sistema de diseño en un único punto y permite implementar un modo oscuro o variantes de tema sin modificar los componentes.

7.4.4 Layout de autenticación en dos paneles

La pantalla de login divide el espacio en dos mitades en desktop: un panel izquierdo de branding con gradiente de marca y el logotipo de Nexly, y un panel derecho con el formulario de acceso. En mobile el panel de branding desaparece y el logotipo se muestra dentro de la tarjeta del formulario. Esta decisión refuerza la identidad visual en el primer contacto del usuario con la aplicación sin penalizar la usabilidad en dispositivos pequeños.

7.4.5 Onboarding multi-paso con barra de progreso

El registro desencadena un modal de onboarding dividido en tres pasos secuenciales: foto de perfil, información personal y privacidad. Una barra de progreso visual indica en todo momento cuántos pasos quedan. El usuario

puede saltar el proceso completo mediante el botón "Completar después" sin perder acceso a la aplicación. Esta decisión mejora la tasa de perfiles completos al presentar la configuración inicial como una tarea guiada y breve en lugar de un formulario largo.

7.4.6 Skeleton loading

El panel de sugerencias de usuarios muestra placeholders animados (animate-pulse) mientras se cargan los datos desde la API. Esto evita saltos de layout y comunica al usuario que el contenido está en camino, sin mostrar spinners centrales que bloqueen la interfaz.

7.4.7 Fallback de avatar

Cuando un usuario no tiene foto de perfil, o cuando la imagen falla al cargar, se muestra un círculo con la inicial de su nombre completo sobre un fondo de color de marca. Esta solución garantiza que las tarjetas de usuario mantienen su aspecto visual completo en cualquier estado de los datos.

7.4.8 Variantes de PostCard

El componente PostCard acepta una prop variant con valores normal y hero. La variante hero aplica mayor padding, tamaño de texto base más grande y una altura máxima de imagen ampliada (max-h-[28rem]), reservándose para el post destacado en la parte superior del feed. El resto de publicaciones usa la variante normal. Esto crea jerarquía visual sin duplicar la lógica del componente.

7.5 Usabilidad

El diseño de Nexly aplica las diez heurísticas de Nielsen. A continuación se detalla cómo se ha abordado cada principio en la implementación real.

7.5.1. Visibilidad del estado del sistema

Los badges numéricos en los iconos de navegación muestran en tiempo real el número de notificaciones y mensajes no leídos. Los botones de formulario cambian su texto durante la operación ("Iniciando sesión...", "Guardando...") e incluyen un estado disabled. La barra de progreso del onboarding indica el paso actual.

7.5.2. Coincidencia con el mundo real

Los iconos del sidebar son universalmente reconocibles: casa para Inicio, siluetas de personas para Comunidades, burbuja de diálogo para Chat, campana para Notificaciones.

7.5.3. Control del usuario y libertad

El onboarding ofrece "Completar después" para salir sin perder el acceso. El buscador del panel lateral se limpia con la tecla Escape (`onKeyDown={(e) => e.key === 'Escape' && limpiar()}}`). Los modales de creación y edición tienen botón de cancelar explícito.

7.5.4. Consistencia y estándares

El estado activo de la navegación usa la misma clase CSS `active` en todos los `NavLink`. El mismo componente `PostCard` se usa en el feed general y en las páginas de comunidad. El sistema de tokens CSS garantiza coherencia visual en toda la aplicación.

7.5.5. Prevención de errores

Los campos del formulario de login usan `type="email"` y `type="password"` con atributo `required`, delegando la validación de formato al navegador antes de enviar la petición. El contador de caracteres de la biografía vira a color de error cuando supera los 140 de los 160 permitidos.

7.5.6. Reconocimiento antes que recuerdo

Todos los items del sidebar muestran icono y etiqueta de texto simultáneamente. Las acciones sobre posts son siempre visibles sin necesidad de interacción previa.

7.5.7. Flexibilidad y eficiencia

El panel de descubrimiento incluye un buscador siempre visible en desktop para búsquedas rápidas sin cambiar de pantalla. El acceso directo al perfil del usuario activo está disponible en la parte inferior del sidebar.

7.5.8. Diseño estético y minimalista

El panel de descubrimiento se oculta en la sección de chat para no fragmentar la atención en una tarea de comunicación directa. Los contadores de reacciones y comentarios solo aparecen si tienen valor mayor que cero.

7.5.9. Ayuda al usuario a reconocer y recuperarse de errores

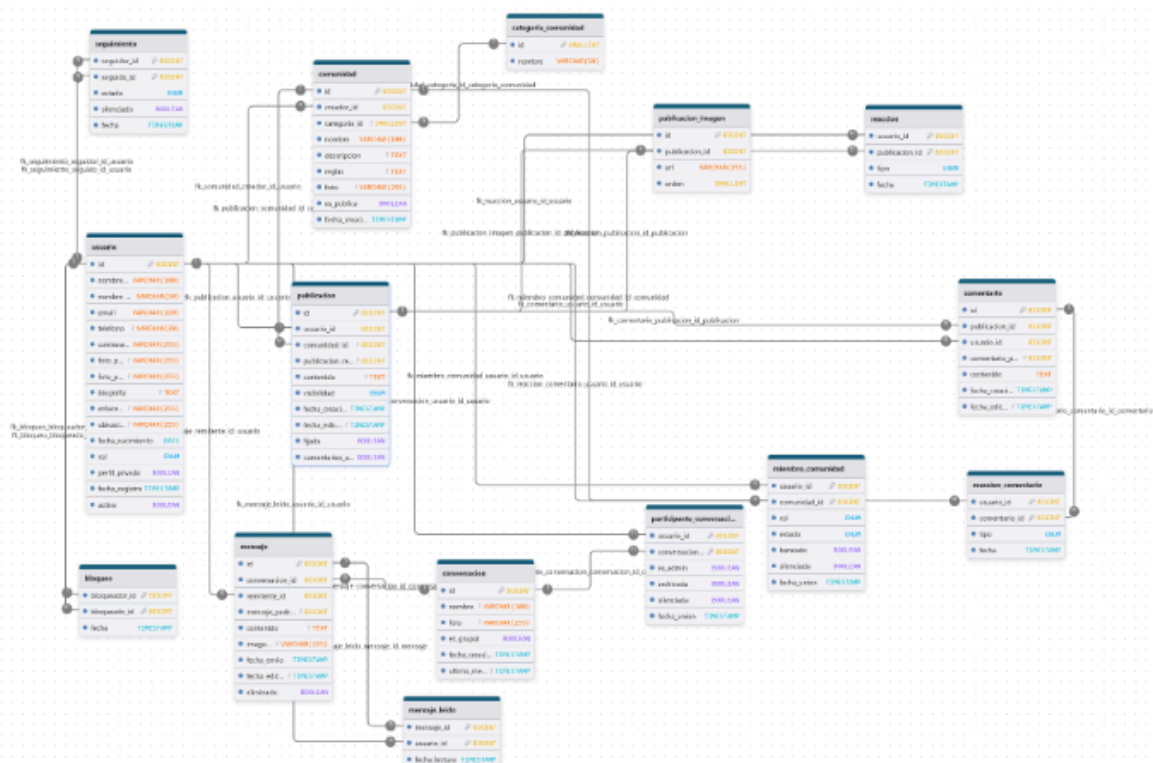
El login distingue entre credenciales incorrectas y cuenta no verificada, ofreciendo en este segundo caso un botón directo para reenviar el correo de verificación. Los errores de carga de imagen en el onboarding se comunican con un mensaje específico.

7.5.10. Ayuda y documentación

El onboarding guía al usuario recién registrado a través de los pasos de configuración inicial. Los campos de formulario incluyen placeholders descriptivos que ejemplifican el formato esperado.

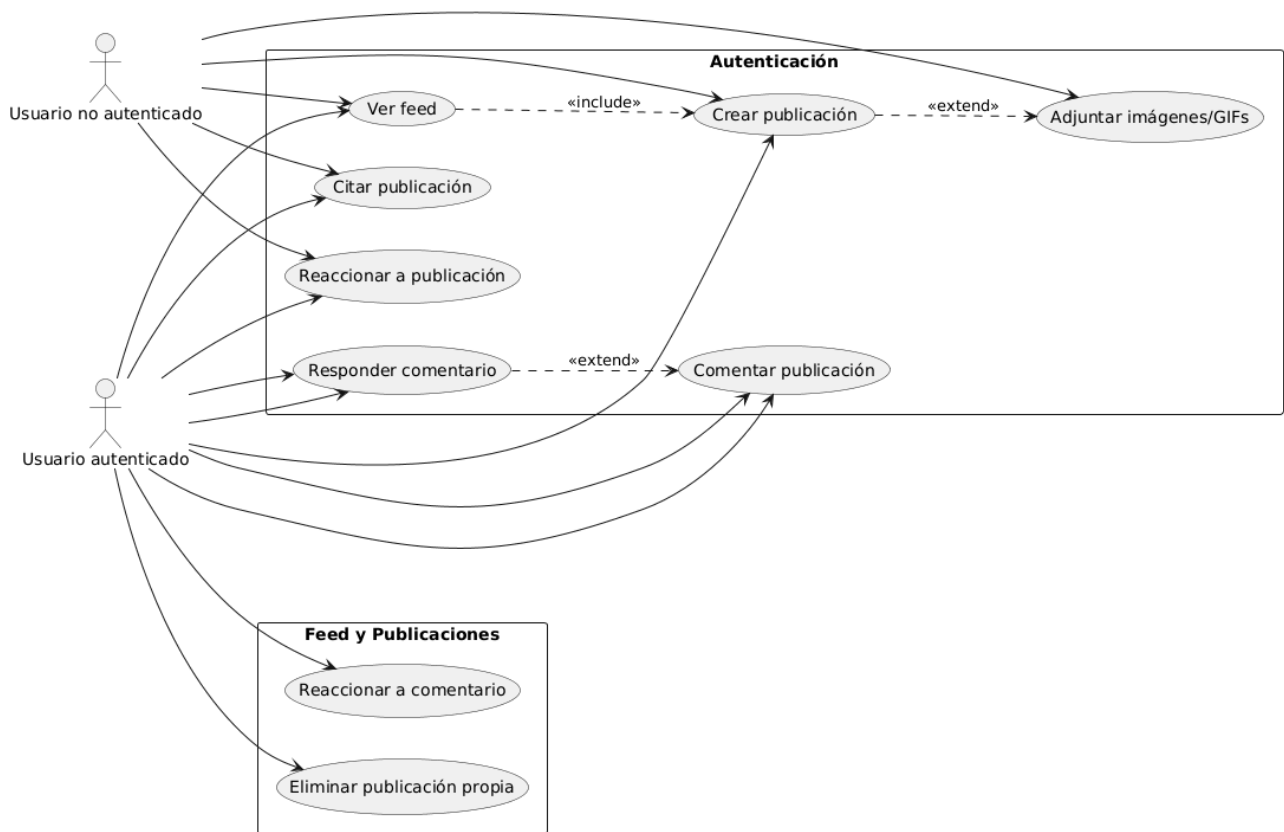
8. Diagramas

8.1. E-R de la(s) base(s) de datos

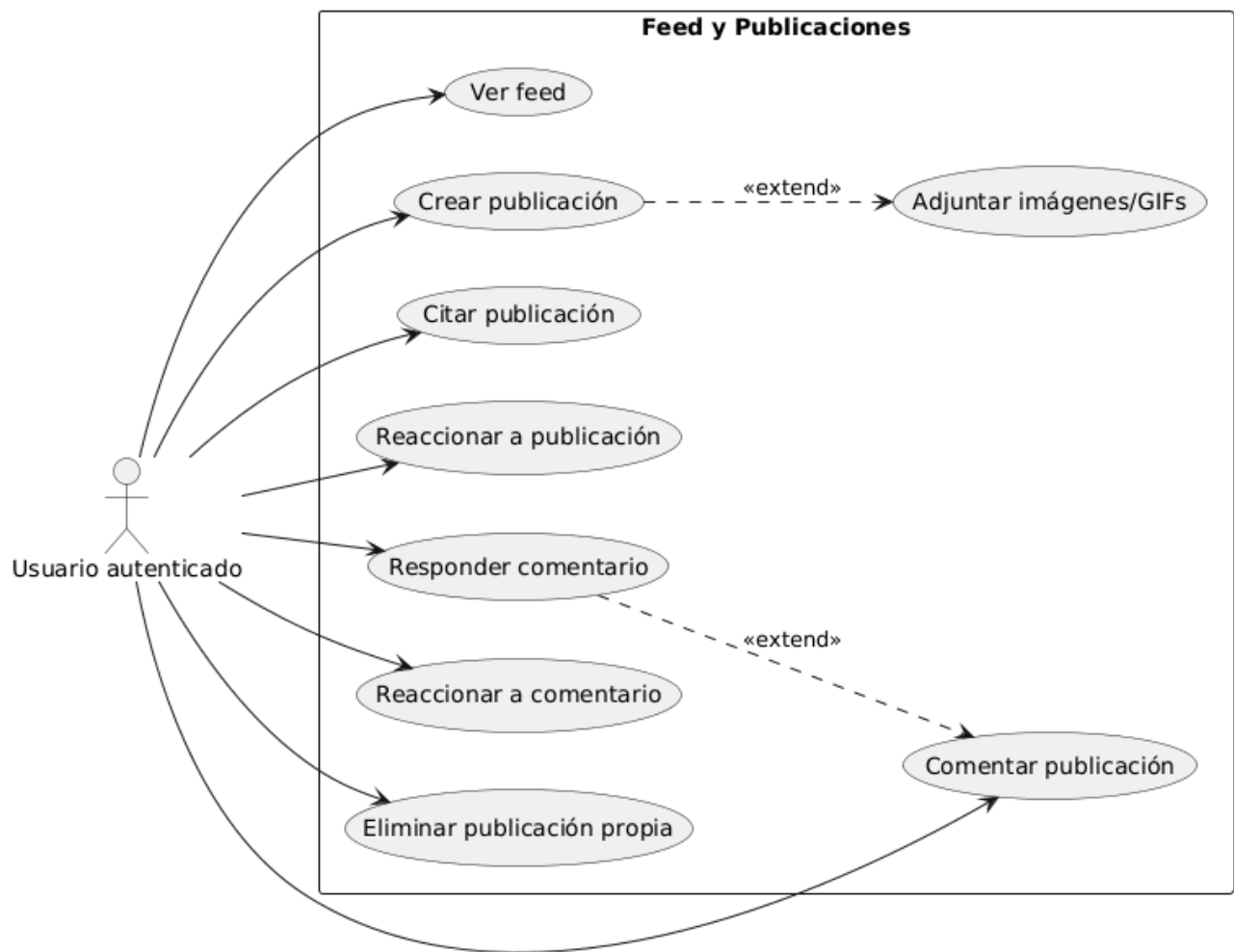


8.2. Diagrama de Casos de Uso

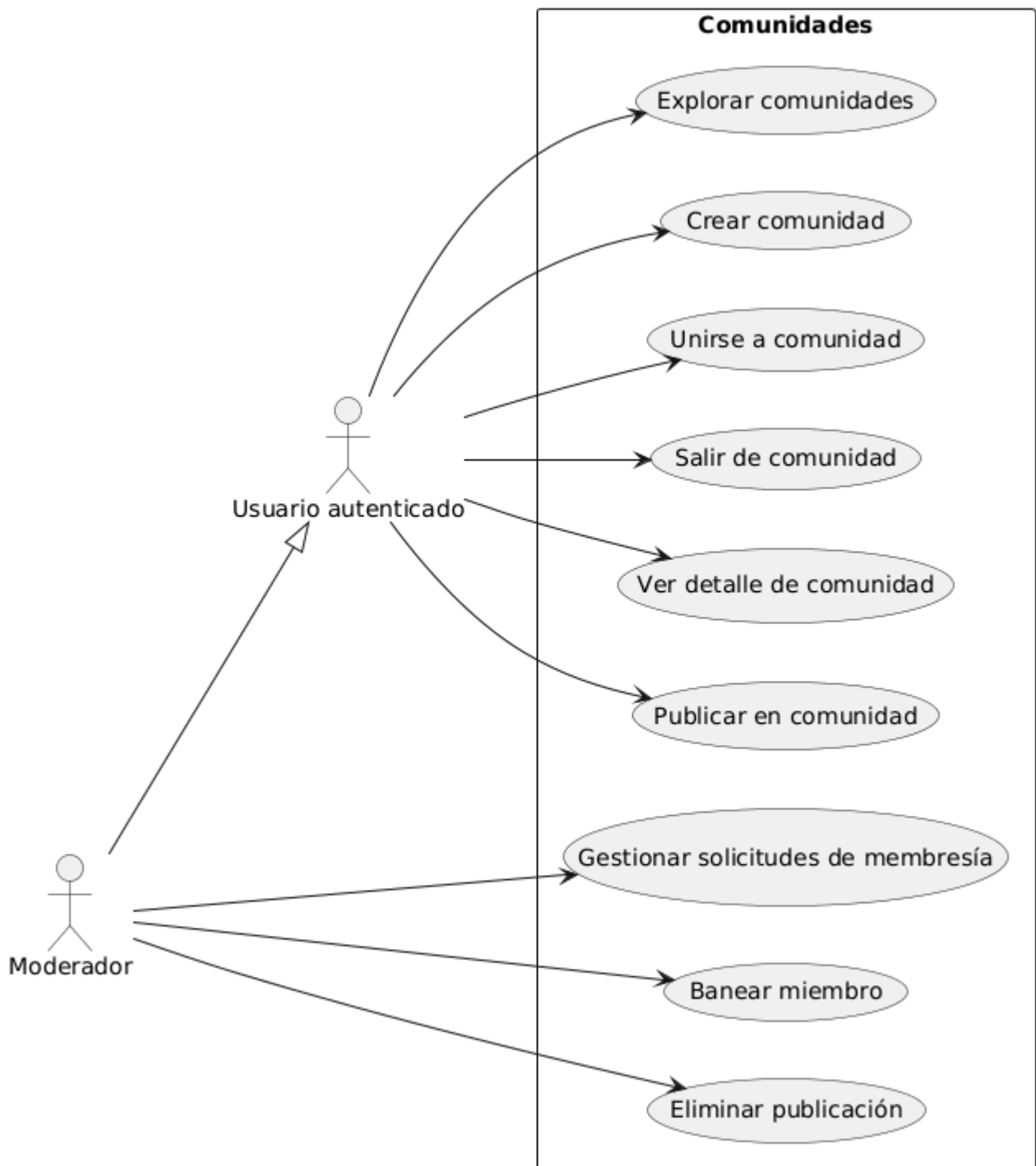
8.2.1. Módulo de autenticación.



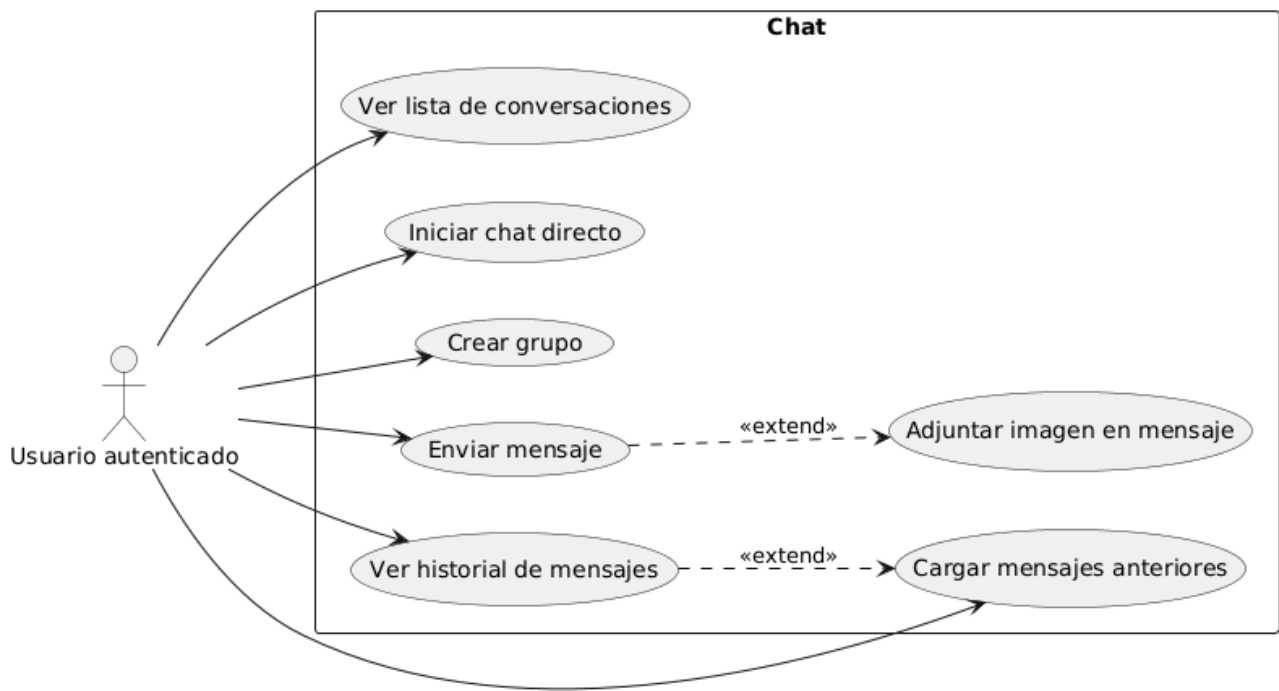
8.2.2. Módulo de Feed y Publicaciones.



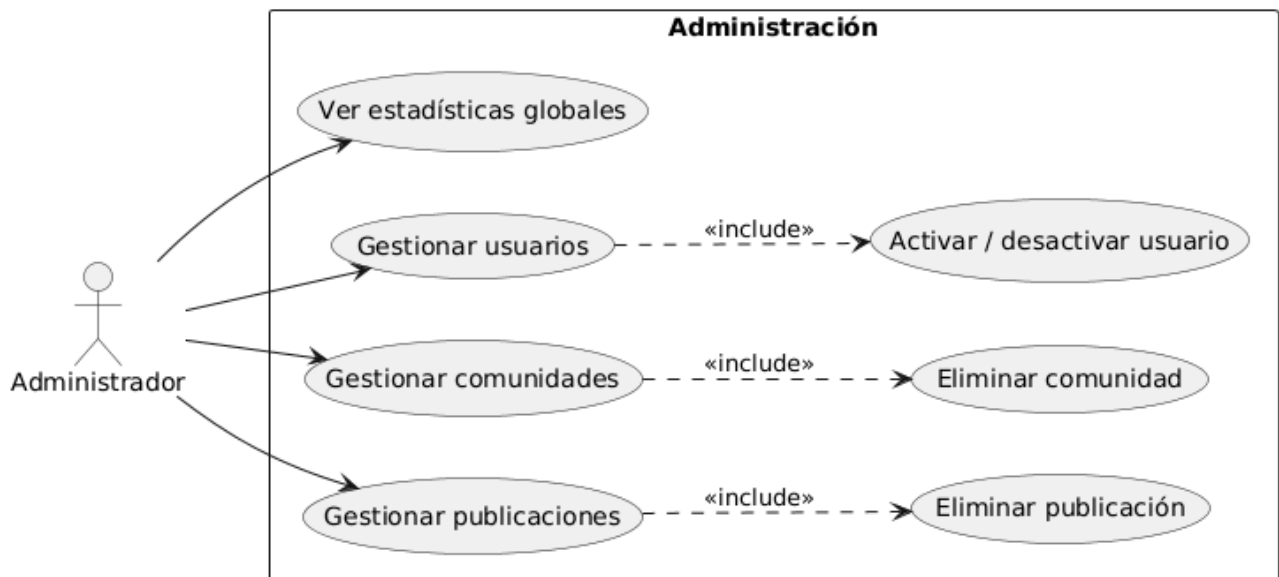
8.2.3. Módulo de Comunidades



8.2.4. Módulo de Chat



8.2.5. Módulo de Administrador

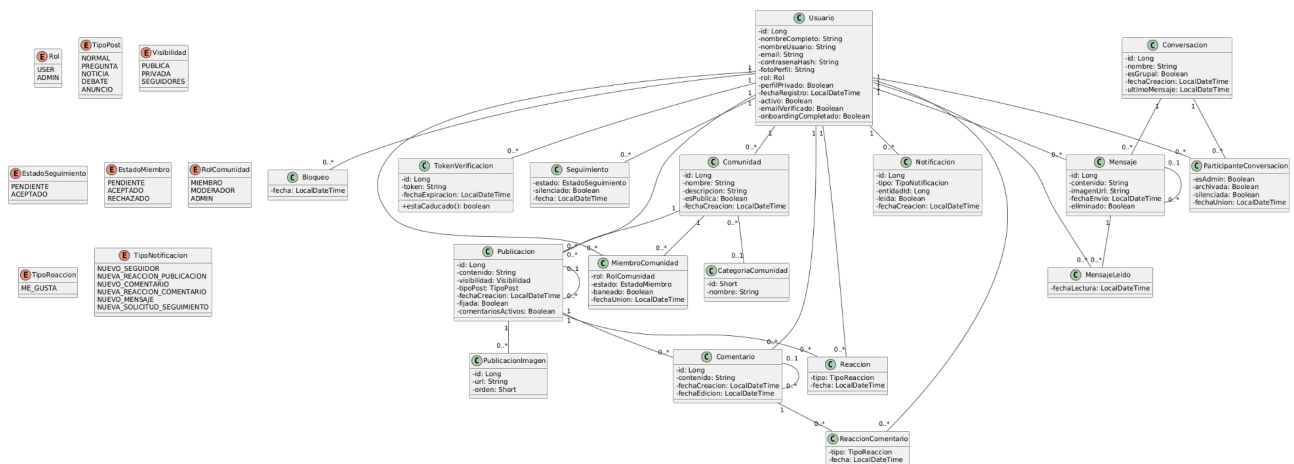


8.2.6. Explicación de los casos de uso

El sistema define cuatro actores con herencia en cadena. El usuario no autenticado solo puede interactuar con el subsistema de autenticación. Tras el

registro, el sistema envía un correo de verificación; hasta que el token es validado, el acceso queda bloqueado (HTTP 403). Una vez autenticado, el usuario accede al onboarding guiado (tres pasos: foto, información personal, privacidad) antes de llegar al feed. El moderador es un usuario con rol MODERADOR o ADMIN dentro de una comunidad concreta; su privilegio no es global sino limitado a esa comunidad. El administrador tiene un rol ADMIN a nivel de sistema, con acceso al panel /admin que permite supervisar y gestionar todos los usuarios, comunidades y publicaciones de la plataforma. La relación <<include>> se usa cuando un caso de uso siempre invoca a otro (registrarse siempre desencadena la verificación de email). La relación <<extend>> indica comportamiento opcional (citar una publicación extiende el caso de crear publicación, pero no es obligatorio).

8.3 Diagrama de clases



8.3.1. Explicación del diagrama de clases

El diagrama refleja el modelo de dominio real implementado con JPA/Hibernate en el backend Spring Boot.

Cada clase corresponde directamente a una tabla de la base de datos y sus anotaciones @Entity. Usuario es la clase central del sistema: participa en

catorce asociaciones distintas. Sus relaciones reflexivas se modelan a través de las clases de asociación Seguimiento y Bloqueo. La decisión de usar clases de asociación independientes (en lugar de tablas de unión simples) se justifica por la necesidad de almacenar atributos propios en la relación:

Seguimiento guarda el estado de la solicitud y si el seguidor ha silenciado al seguido;

MiembroComunidad guarda el rol dentro de la comunidad, el estado de la solicitud de membresía y si el usuario ha sido baneado.

Publicacion implementa la autocita mediante una asociación reflexiva a publicacion_ref_id.

Cuando este campo no es nulo, la publicación es una cita de otra, y el frontend renderiza la QuoteCard embebida. Esto permite citas en cadena sin límite de profundidad aunque la interfaz solo muestra un nivel. Comentario también es autoasociativo mediante comentario_padre_id, modelando respuestas a comentarios.

Los enums TipoPost y Visibilidad sobre Publicacion, y RolComunidad y EstadoMiembro sobre MiembroComunidad, son tipos que en Java se mapean como EnumType.STRING en la base de datos, lo que garantiza legibilidad en las consultas SQL directas.

El módulo de chat separa la conversación de sus participantes y mensajes en tres clases: Conversacion almacena los metadatos del hilo, ParticipanteConversacion registra quién pertenece al hilo y con qué privilegios (esAdmin para grupos), y Mensaje contiene el contenido.

La clase MensajeLeido implementa el patrón de confirmación de lectura individual: un registro por cada par (mensaje, usuario) que ha leído ese mensaje.

Notificacion usa un diseño polimórfico con el campo entidadId de tipo genérico Long en lugar de múltiples claves foráneas opcionales. El tipo de la entidad referenciada se infiere del campo tipo (enum TipoNotificacion). Este patrón

simplifica el esquema a costa de no poder aplicar integridad referencial a nivel de base de datos sobre entidadId.

9. Desarrollo del proyecto

9.1 Arquitectura general

Nexly sigue una arquitectura cliente-servidor desacoplada. El frontend y el backend son proyectos independientes que se comunican a través de una API REST y un canal WebSocket para eventos en tiempo real.

El despliegue local se orquesta con Docker Compose: un contenedor MySQL, un contenedor Mailpit para emails en desarrollo, y el backend Spring Boot. El frontend corre con el servidor de desarrollo de Vite en el puerto 5173.

9.2 Estructura del Backend

El backend sigue la arquitectura en capas estándar de Spring Boot:

Controllers → Reciben la petición HTTP y delegan al servicio

Services → Lógica de negocio, transacciones

Repositories → Acceso a datos (Spring Data JPA)

Models → Entidades JPA mapeadas a tablas MySQL

DTOs → Objetos de transferencia de datos (entrada y salida)

Mappers → Conversión entre entidades y DTOs

Security → JwtFilter, SecurityConfig, OAuth2

WebSocket → ChatController (STOMP)

La capa de seguridad intercepta todas las peticiones mediante JwtFilter, que extrae el token JWT de la cookie HttpOnly, valida la firma HMAC-SHA256 y

carga el `userId` y `rol` en el contexto de seguridad de Spring. El endpoint `/api/auth/**`, `/oauth2/**` y `/ws/**` son las únicas rutas públicas.

9.3 Desarrollo por módulos

9.3.1 Autenticación

El sistema soporta dos flujos de autenticación: Registro con email: El usuario envía nombre, email y contraseña.

El backend hashlea la contraseña con BCrypt, persiste el usuario con `emailVerificado = false` y genera un `TokenVerificacion` de 64 caracteres que caduca en 24 horas.

Mailpit recibe el email con el enlace de verificación.

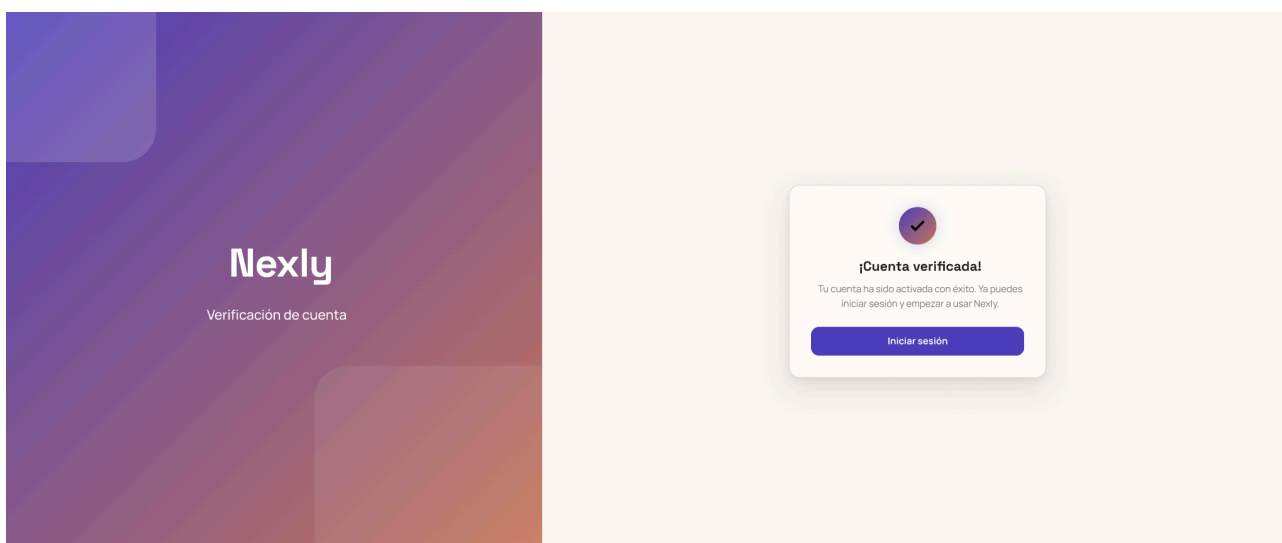
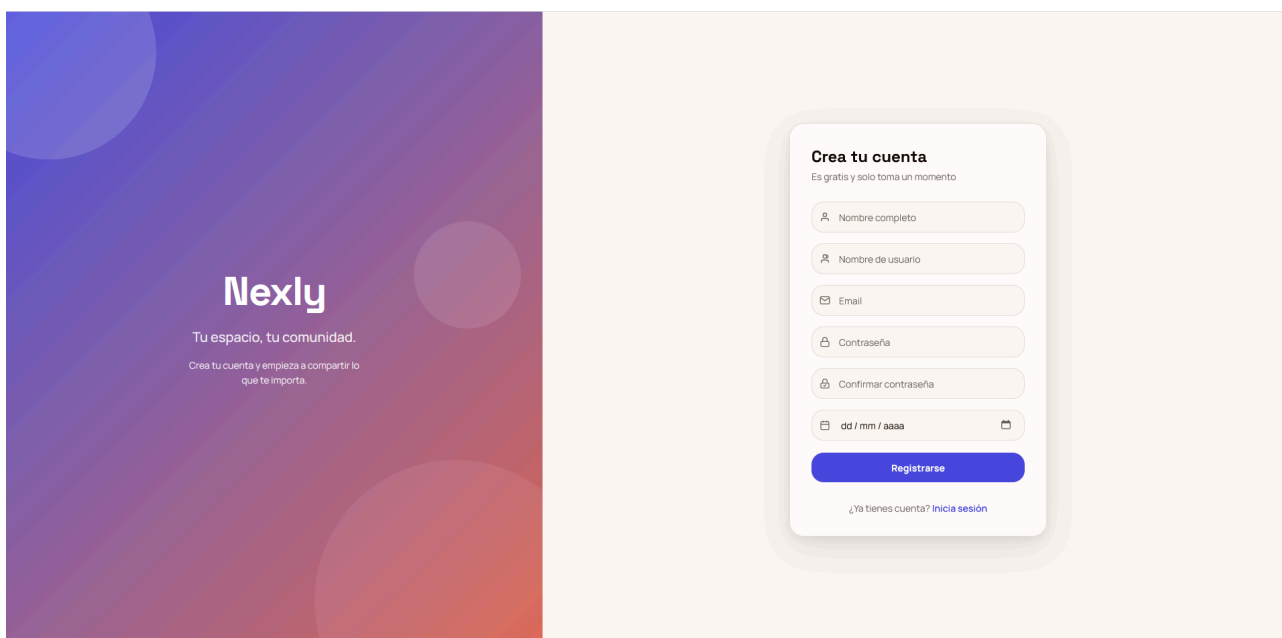
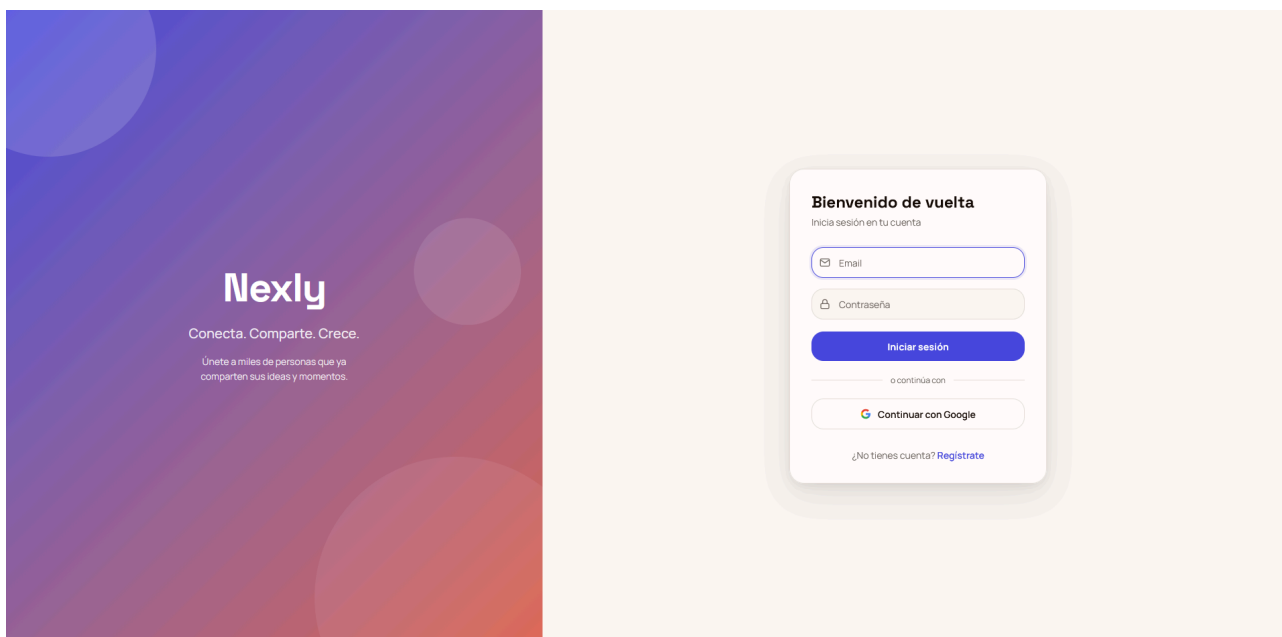
Hasta que el token es consumido, el login devuelve HTTP 403 con el cuerpo `{"error": "EMAIL_NO_VERIFICADO"}`, lo que permite al frontend ofrecer el reenvío del correo sin revelar si el email existe en el sistema.

Login con Google OAuth2: Spring Security gestiona el flujo OAuth2 Authorization Code.

El `OAuth2SuccessHandler` busca el usuario por `oauthId` o email; si no existe, lo crea con `emailVerificado = true` (Google ya lo garantiza).

En ambos casos genera el JWT y lo escribe como cookie `HttpOnly` con `SameSite=Strict`. El JWT se envía siempre en cookie, nunca en `localStorage`, evitando ataques XSS que robarían el token.

La cookie tiene `HttpOnly = true` (inaccesible desde JavaScript) y `maxAge = 86400` segundos (24 horas).



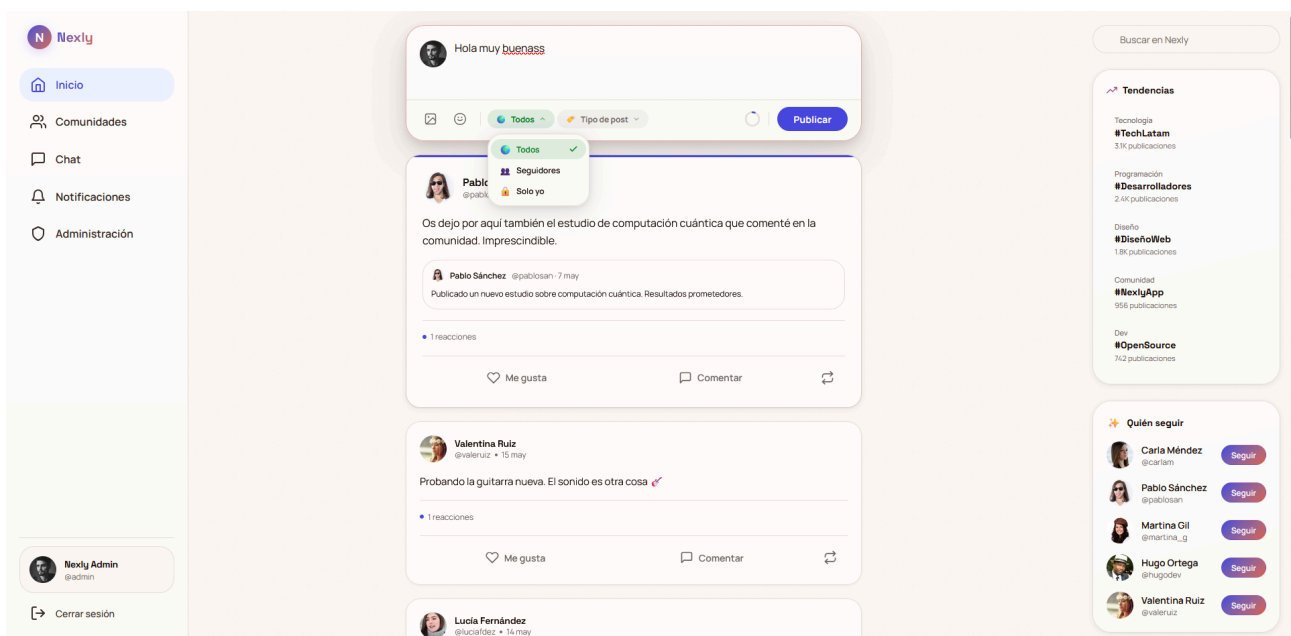
9.3.2 Feed y publicaciones

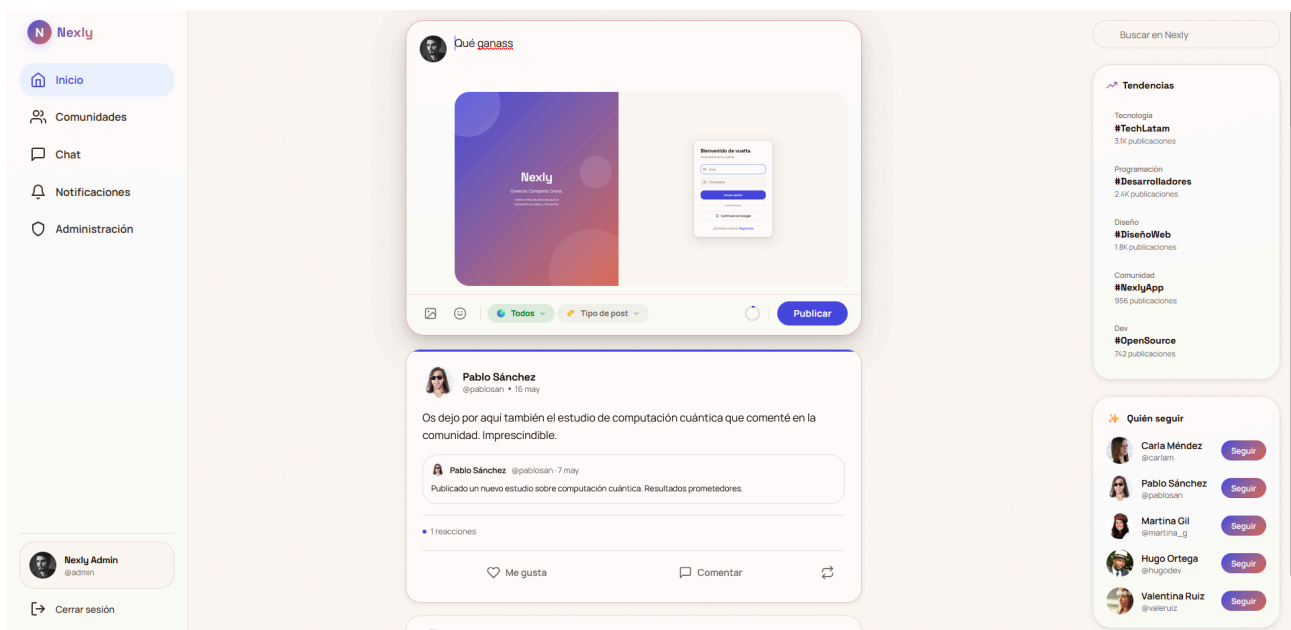
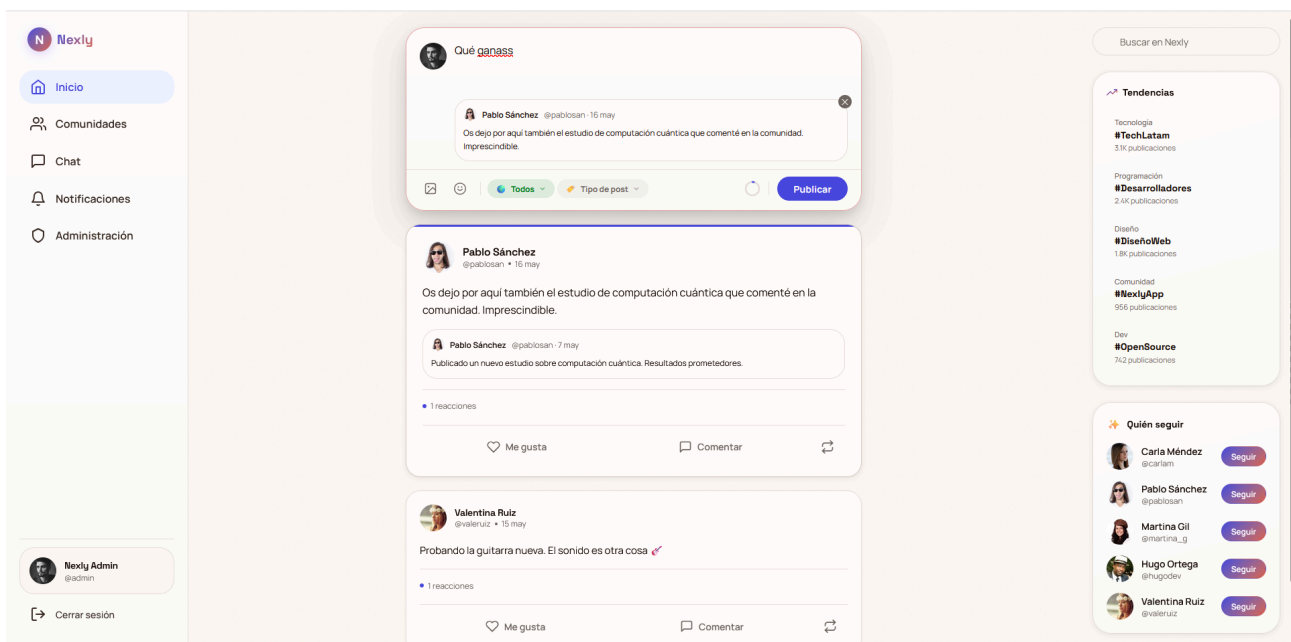
El feed carga publicaciones paginadas (10 por página) de los usuarios seguidos por el usuario autenticado. El endpoint GET /api/publicaciones/feed filtra por seguidor_id y respeta la visibilidad de cada publicación (PUBLICA, PRIVADA, SEGUIDORES).

El componente CreatePost permite: Escribir texto libre Seleccionar tipo de publicación: NORMAL, PREGUNTA, NOTICIA, DEBATE, ANUNCIO (mostrado como FlairBadge) Adjuntar hasta 3 imágenes o GIFs (se suben a Cloudinary antes de enviar el formulario)

Seleccionar visibilidad Publicar en una comunidad de la que se es miembro La función de cita (QuoteCard) envía el publicacionRefId de la publicación citada.

El backend persiste la referencia en publicacion_ref_id y el frontend anida el QuoteCard dentro del PostCard de la publicación nueva. Las reacciones usan clave primaria compuesta (usuario_id, publicacion_id), lo que hace el toggle atómico: si la fila ya existe se elimina, si no existe se crea.





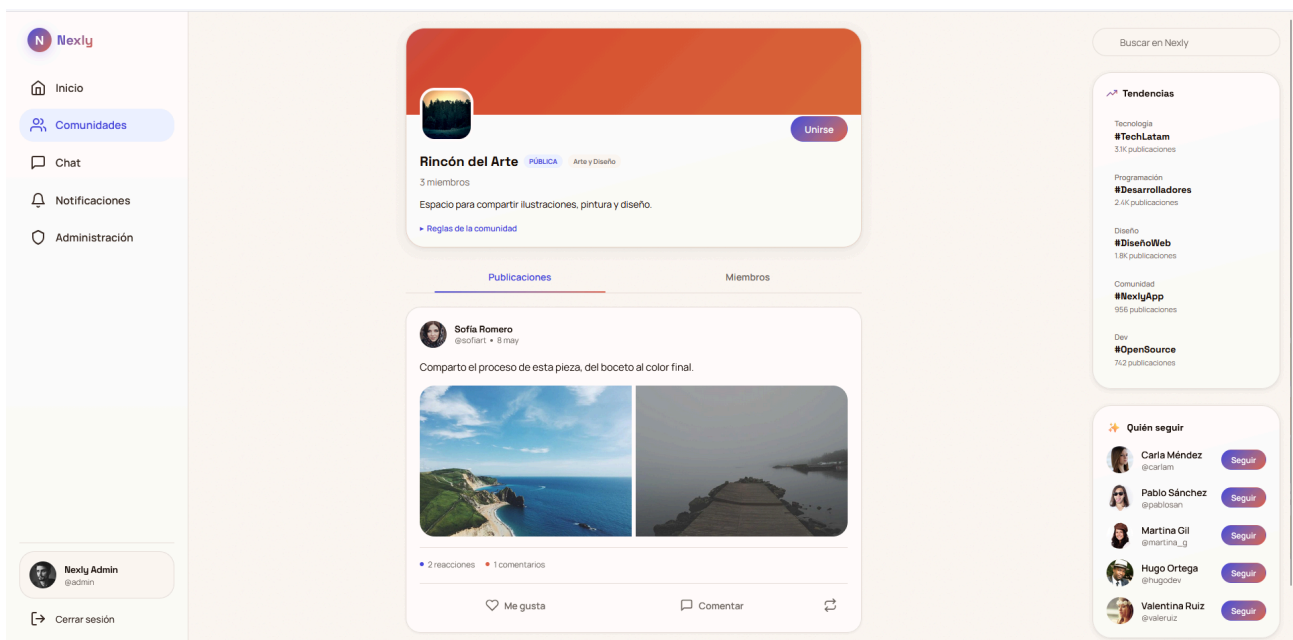
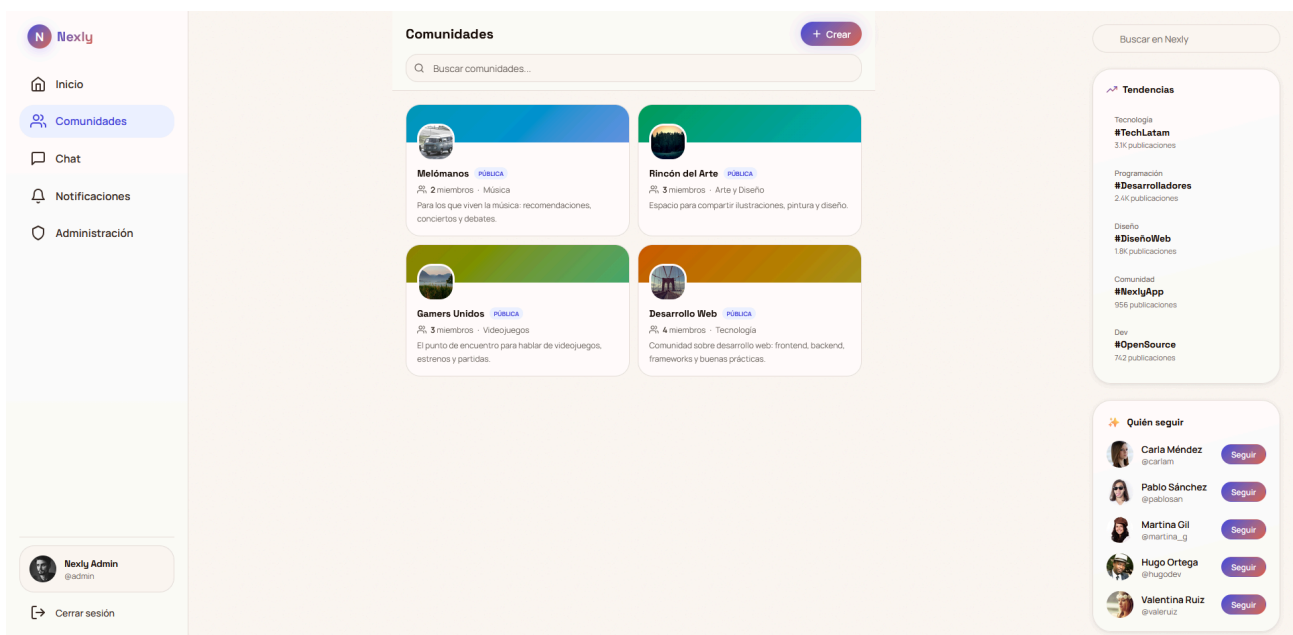
9.3.3 Comunidades

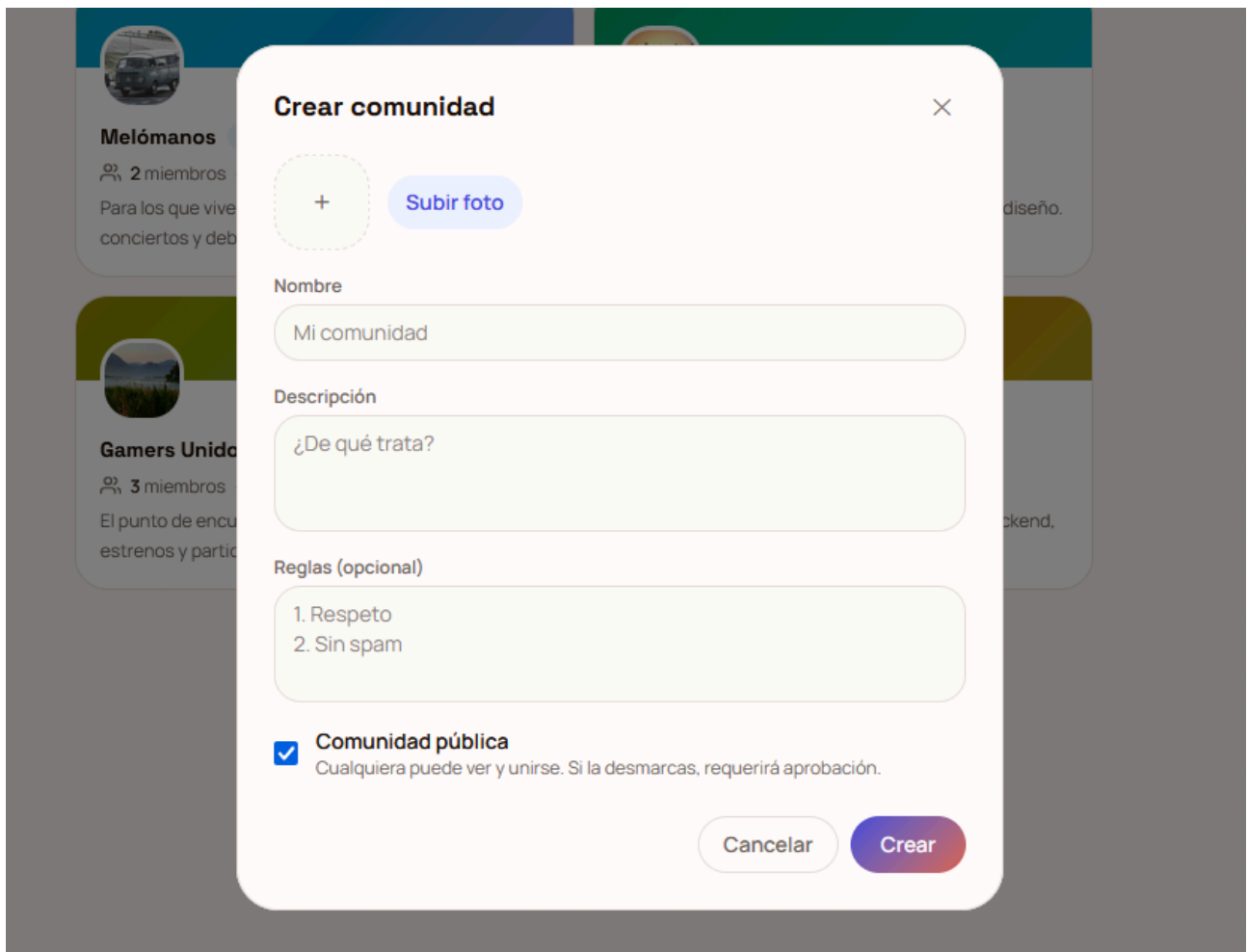
Una comunidad puede ser pública o privada. La diferencia afecta al flujo de unión:

Pública: `unirse()` crea `MiembroComunidad` con estado = `ACEPTADO` directamente.

Privada: `unirse()` crea `MiembroComunidad` con estado = `PENDIENTE`. El moderador/admin ve la solicitud en la lista de pendientes y puede aceptarla o rechazarla. El creador de una comunidad recibe automáticamente el rol `ADMIN` de

esa comunidad. El rol ADMIN de la comunidad es distinto del rol ADMIN global del sistema: un ADMIN de comunidad solo tiene privilegios dentro de esa comunidad concreta. Las reglas de negocio más relevantes implementadas en ComunidadService: El nombre de comunidad debe ser único en todo el sistema. El ADMIN de comunidad no puede salir de ella sin transferir el rol primero. Solo el ADMIN de comunidad puede cambiar roles; no puede asignar el rol ADMIN directamente (debe usar la operación de transferencia). No se puede banear al administrador de la comunidad.

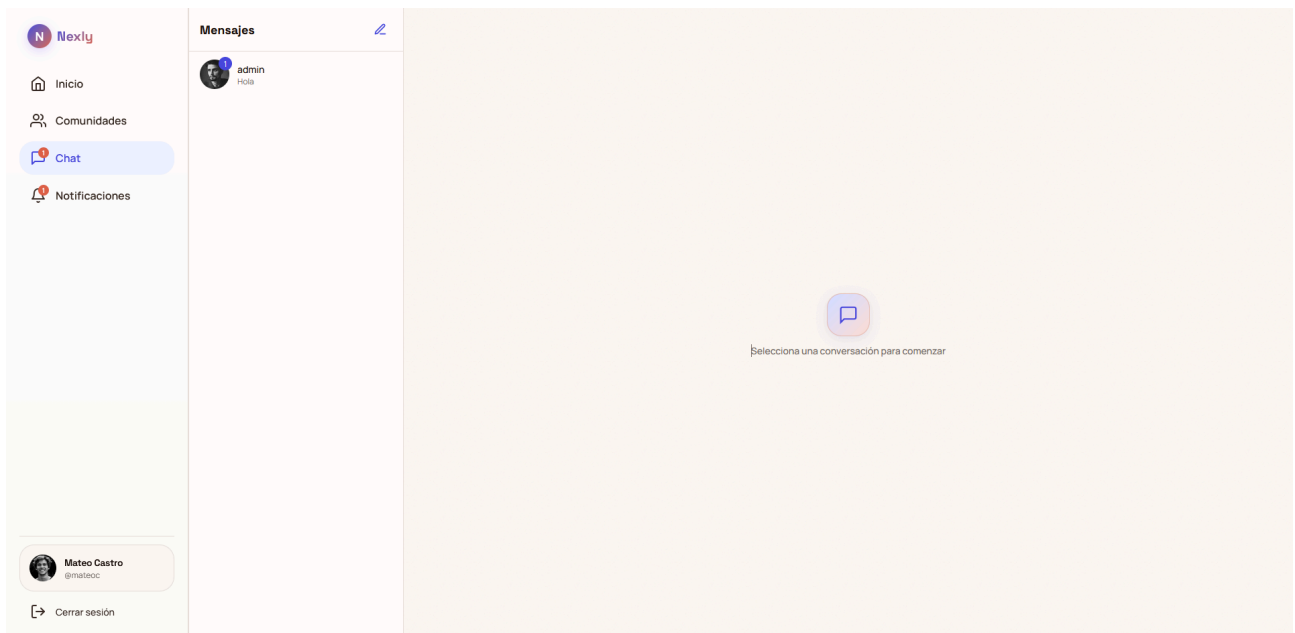
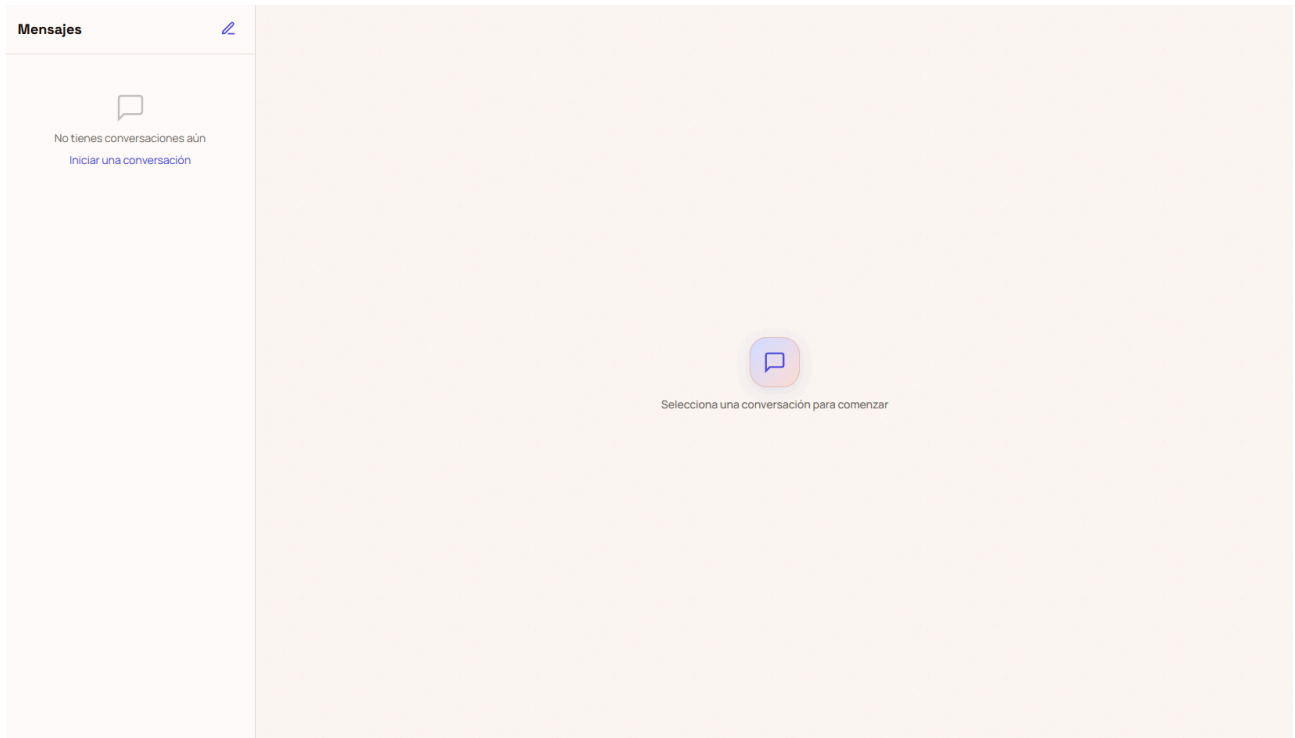


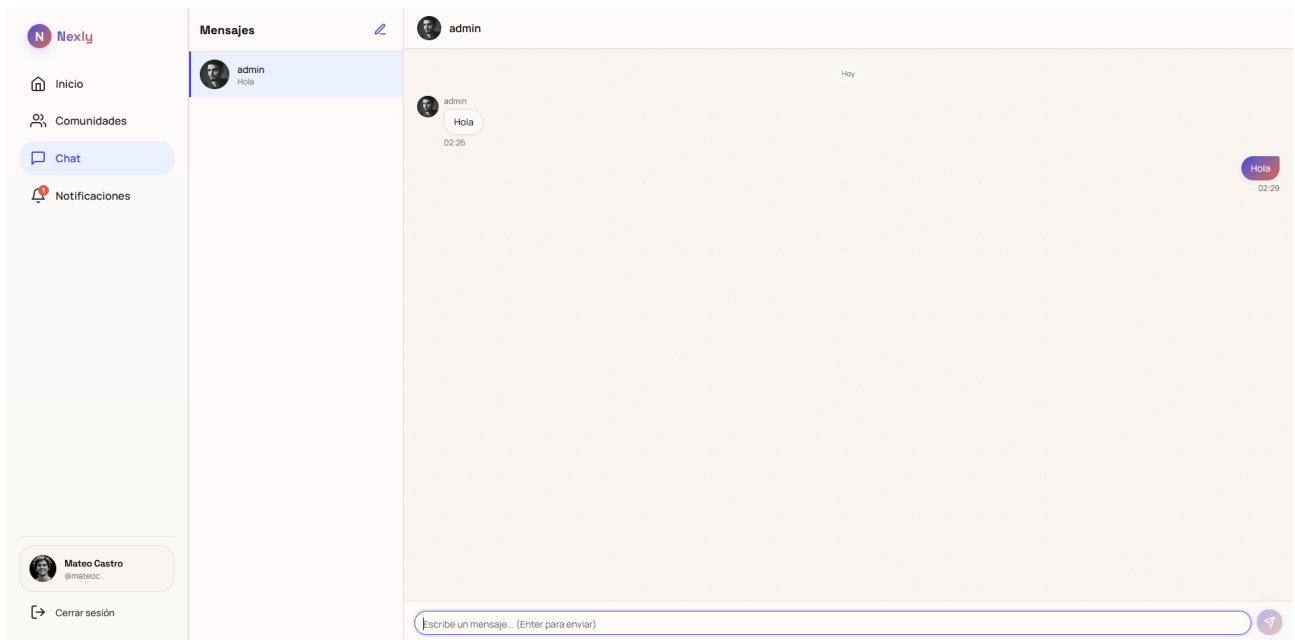


9.3.4 Chat en tiempo real

El chat usa WebSocket con el protocolo STOMP sobre SockJS. El flujo completo es: El frontend conecta al endpoint `/ws` al cargar `AppLayout`. Para enviar un mensaje, el cliente publica en `/app/chat/{conversacionId}`. El `ChatController` del backend recibe el mensaje, llama a `MensajeService.enviar()`, persiste el mensaje en base de datos y emite el DTO resultante a todos los participantes de la conversación mediante `messagingTemplate.convertAndSendToUser(userId, "/queue/mensajes", dto)`. Paralelamente, `NotificacionService.emitir()` envía una notificación de tipo `NUEVO_MENSAJE` a todos los participantes excepto el remitente. El scroll del chat se gestiona con `useLayoutEffect` en `ChatWindow`: detecta si el mensaje nuevo es un `append` (llegó un mensaje nuevo) o un `prepend`

(se cargaron mensajes anteriores con "cargar más"), y ajusta scrollTop en consecuencia sin producir saltos visuales. Las conversaciones pueden ser directas (dos participantes) o grupales (esGrupal = true). Al iniciar un chat directo, el sistema busca primero si ya existe una conversación entre los dos usuarios para no crear duplicados.



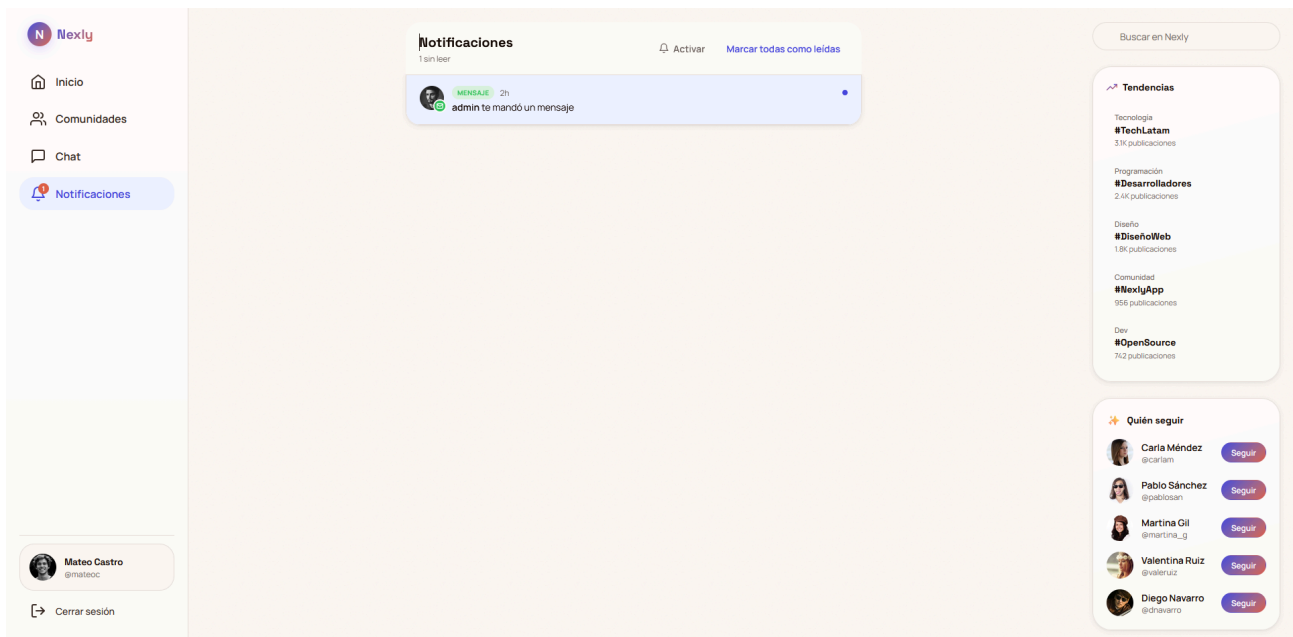


9.3.5 Notificaciones en tiempo real

Las notificaciones se emiten desde `NotificacionService.emitir()`, que es llamado por los demás servicios tras cada acción relevante:

- Un usuario te sigue (perfil público): `NUEVO_SEGUIDOR`
- Un usuario solicita seguirte (perfil privado):
`NUEVA_SOLICITUD_SEGUIMIENTO`
- Alguien reacciona a tu publicación: `NUEVA_REACCION_PUBLICACION`
- Alguien comenta tu publicación: `NUEVO_COMENTARIO`
- Alguien reacciona a tu comentario: `NUEVA_REACCION_COMENTARIO`
- Recibes un mensaje en un chat: `NUEVO_MENSAJE`

Además del `WebSocket`, el servicio llama a `WebPushService.enviar()` de forma asíncrona para disparar notificaciones push del navegador, incluso si el usuario no tiene la pestaña abierta. El badge de notificaciones del sidebar se actualiza en tiempo real sin recargar la página.



9.4 API REST — principales endpoints

- POST → /api/auth/register → Registro de usuario
- POST → /api/auth/login → Login con email/contraseña
- POST → /api/auth/verificar → Verificar token de email
- POST → /api/auth/logout → Cerrar sesión (borra cookie)
- GET → /api/publicaciones/feed → Feed paginado del usuario
- POST → /api/publicaciones → Crear publicación
- POST → /api/publicaciones/{id}/reacción → Toggle reacción
- GET → /api/comunidades → Listado de comunidades públicas
- POST → /api/comunidades → Crear comunidad
- POST → /api/comunidades/{id}/unirse → Unirse a comunidad
- GET → /api/usuario/{id} → Perfil de usuario
- POST → /api/usuario/{id}/seguir → Seguir usuario
- GET → /api/mensajes/conversaciones → Lista de conversaciones
- POST → /api/mensajes/directa/{userId} → Iniciar chat directo
- GET → /api/notificaciones → Notificaciones paginadas
- PATCH → /api/notificaciones/leer-todas → Marcar todas como leídas

GET → /api/admin/stats → Estadísticas globales (solo ADMIN)

PATCH → /api/admin/usuarios/{id}/estado → Activar/desactivar usuario (solo ADMIN)

10. Forma jurídica y trámites de constitución

Forma jurídica elegida: Sociedad Limitada Unipersonal (SLU)

Para Nexly se propone la Sociedad Limitada Unipersonal (SLU), variante de la Sociedad de Responsabilidad Limitada (SRL) pensada para un único socio fundador.

Justificación:

Responsabilidad → Ilimitada (patrimonio personal) → Limitada al capital aportado

Capital mínimo → 0 € → 3.000 € (puede ser en bienes)

Coste de constitución → Muy bajo (~300 €) → Medio (~600–1.000 €)

Imagen frente a inversores → Limitada → Profesional y escalable

Tributación → IRPF (tipo marginal) → Impuesto de Sociedades (25 %)

La naturaleza del proyecto →> plataforma digital con datos de usuarios, posible búsqueda de inversión y crecimiento escalable →> hace que la responsabilidad limitada sea determinante: si Nexly incurriese en deudas o reclamaciones (p. ej. por protección de datos), el patrimonio personal del fundador quedaría protegido.

Además, la SLU facilita incorporar socios o inversores en el futuro sin cambiar la forma jurídica. El autónomo quedaría descartado principalmente por la responsabilidad ilimitada, que expone el patrimonio personal ante cualquier contingencia legal o económica del negocio.

10.1. Principales trámites de constitución

10.1.1 Certificación negativa de denominación social

Solicitud al Registro Mercantil Central para reservar el nombre "Nexly" y comprobar que no existe otra sociedad con él.

10.1.2 Apertura de cuenta bancaria y desembolso del capital

Ingreso de los 3.000 € mínimos en una cuenta a nombre de la sociedad en constitución; el banco expide un certificado de depósito.

10.1.3. Elaboración de los Estatutos Sociales

Documento que regula el objeto social, el capital, los órganos de gobierno y las normas de funcionamiento de la sociedad.

10.1.4. Escritura pública ante notario

Firma de la escritura de constitución con los estatutos ante notario. Coste aproximado: 150–300 €.

10.1.5. Obtención del NIF provisional

Tramitación en la Agencia Tributaria (modelo 036) para obtener el Número de Identificación Fiscal que permite operar.

10.1.6. Liquidación del Impuesto de Transmisiones Patrimoniales (ITP) — modalidad Operaciones Societarias

Actualmente está exento para sociedades de nueva creación, pero el trámite sigue siendo obligatorio.

10.1.7. Inscripción en el Registro Mercantil Provincial

Inscripción de la escritura en el Registro Mercantil de la provincia correspondiente. Otorga personalidad jurídica plena a la sociedad.

10.1.8. Alta en la Agencia Tributaria

Presentación del modelo 036/037: alta en el censo de empresarios, declaración de inicio de actividad y elección del régimen de IVA.

10.1.9 Alta en la Seguridad Social — RETA

El socio administrador deberá darse de alta en el Régimen Especial de Trabajadores Autónomos (cuota reducida de 80 €/mes el primer año en 2025).

10.1.10. Licencias y obligaciones sectoriales

Dado que Nexly trata datos personales, es obligatorio cumplir con el Reglamento General de Protección de Datos (RGPD) y redactar la política de privacidad, aviso legal y política de cookies antes del lanzamiento.

Este proceso puede completarse en aproximadamente 2–4 semanas y con un coste total estimado de 800–1.500 € (notaría, registro y tasas), sin contar el capital social.

11. Viabilidad tecno-económica

11.1. Costes de desarrollo (inversión inicial)

El desarrollo de Nexly ha sido realizado íntegramente como Proyecto de Fin de Ciclo, por lo que el coste directo en mano de obra es nulo para el promotor. No obstante, para reflejar el valor económico real del proyecto, se estima el coste equivalente en el mercado laboral español:

- Diseño y arquitectura → 40 h → 15 €/h → 600 €
- Desarrollo backend (Spring Boot, BBDD, seguridad) → 180 h → 15 €/h → 2.700 €
- Desarrollo frontend (React, Zustand, WebSocket) → 160 h → 15 €/h → 2.400 €
- Integración y pruebas → 60 h → 15 €/h → 900 €
- Documentación → 20 h → 15 €/h → 300 €
- Total desarrollo → 460 h → 6.900 €

A esto se suman los costes de constitución de la SLU (~1.000 €) y los de identidad de marca (logo, diseño básico, ~300 €), dando una inversión inicial total aproximada de 8.200 €.

11.2. Costes de infraestructura y operación

Los costes de infraestructura en cloud son especialmente reducidos gracias a los niveles gratuitos de los servicios utilizados y al uso de proveedores europeos económicos.

11.2.1 Escenario A — Lanzamiento / hasta ~500 usuarios activos

Servidor VPS (2 vCPU, 4 GB RAM, 40 GB SSD) → Hetzner CX21 → 6,00 €

Base de datos MySQL (en mismo VPS) → Incluida → 0,00 €

Almacenamiento de imágenes (hasta 25 GB) → Cloudinary Free → 0,00 €

Envío de emails transaccionales (hasta 300/día) → Brevo Free → 0,00 €

Certificado SSL → Let's Encrypt → 0,00 €

CDN y protección DDoS básica → Cloudflare Free → 0,00 €

Nombre de dominio (nexly.app, prorrateo mensual) → Namecheap → 1,50 €

Total mensual → 7,50 €

Total anual → 90,00 €

11.2.2. Escenario B — Crecimiento / 500–10.000 usuarios activos

Servidor VPS (4 vCPU, 8 GB RAM) → Hetzner CX32 → 15,00 €

Base de datos gestionada MySQL → Hetzner DBaaS → 20,00 €

Almacenamiento de imágenes (plan de pago) → Cloudinary Plus → 99,00 €

Envío de emails transaccionales → Brevo Starter → 19,00 €

Monitorización y alertas → Grafana Cloud Free → 0,00 €

Dominio + SSL → Namecheap + Let's Encrypt → 1,50 €

Total mensual → 154,50 €

Total anual → 1.854,00 €

En el escenario B, si el volumen de imágenes es moderado, Cloudinary puede sustituirse por Cloudflare R2 (~0,015 €/GB/mes), reduciendo ese coste a menos de 5 €/mes durante los primeros meses.

11.3. Modelo de monetización y análisis de rentabilidad

Nexly puede generar ingresos mediante un modelo freemium combinado con publicidad:

Nexly Premium → Sin anuncios, distintivo verificado, estadísticas de perfil, más almacenamiento → 4,99 €/mes

Comunidades Premium → Comunidades de pago con acceso restringido a sus administradores → 2,99 €/mes

Publicidad → Publicaciones patrocinadas en el feed (CPM ~1,50 €) → Variable

11.3.1 Punto de equilibrio (break-even)

Con los costes del Escenario A (7,50 €/mes):

2 suscriptores Premium ya cubren los costes operativos mensuales.

Con los costes del Escenario B (154,50 €/mes):

Se necesitan 31 suscriptores Premium (4,99 €) para alcanzar el punto de equilibrio, lo cual es muy alcanzable con una base de 500–1.000 usuarios activos (tasa de conversión freemium media del 2–5 %).

11.4. Tabla resumen — Coste total primer año

Inversión inicial (constitución + marca) → 1.300,00 €

Valor del desarrollo (coste de oportunidad) → 6.900,00 €

Infraestructura año 1 — Escenario A (×12) → 90,00 €

Total primer año (escenario conservador) → 8.290,00 €

Infraestructura año 1 — Escenario B (×12) → 1.854,00 €

Total primer año (escenario crecimiento) → 10.054,00 €

11.5. Conclusión de viabilidad

El proyecto es económicamente viable desde sus primeras fases. Los costes de infraestructura son extremadamente bajos al arranque (menos de 8 €/mes), lo que permite operar sin ingresos durante los meses iniciales con un riesgo financiero mínimo. La plataforma entraría en rentabilidad operativa con una base de usuarios muy reducida, y el principal riesgo no es tecnológico ni económico, sino de captación de usuarios en un mercado con competidores consolidados (Instagram, Reddit, Discord). La diferenciación por comunidades verticales y la identidad de red social en español constituyen la principal ventaja competitiva.

12. Trabajo futuro

Nexly constituye una base sólida y funcional, pero el desarrollo de una red social es un proceso continuo.

A continuación se describen las mejoras e incorporaciones más relevantes que podrían acometerse en versiones futuras, organizadas por área.

12.1 Nuevas funcionalidades

Contenido efímero (Stories)

Publicaciones con caducidad de 24 horas, visibles en la parte superior del feed. Ampliamente demandadas en redes sociales modernas, aumentan la frecuencia de visita y la interacción diaria.

Llamadas de audio y vídeo en el chat

Integración de WebRTC sobre la infraestructura WebSocket ya existente para ofrecer llamadas en tiempo real dentro de las conversaciones privadas y de grupo. La base STOMP ya presente facilita la señalización necesaria.

Encuestas en publicaciones

El commit e05869c refleja que las encuestas fueron eliminadas durante el desarrollo por restricciones de tiempo. Su reimplementación añadiría valor a los tipos de post "pregunta" y "debate", que son los más indicados para ellas.

Edición y reacciones en mensajes de chat

El chat actual permite enviar y leer mensajes, pero no editarlos ni reaccionar a ellos con emojis. Ambas funcionalidades son ya estándar en plataformas de mensajería.

Publicaciones con vídeo

Actualmente solo se admiten imágenes vía Cloudinary. La subida y reproducción de vídeos cortos (con transcodificación en la nube) ampliaría significativamente los tipos de contenido posibles.

Programación de publicaciones

Permitir que los administradores de comunidades y usuarios Premium publiquen contenido en una fecha y hora futura, útil para comunidades con estrategia editorial.

12.2. Mejoras técnicas y de rendimiento

Caché con Redis

Introducir una capa de caché para el feed paginado, los perfiles y los contadores de reacciones. Reduciría la carga sobre MySQL y mejoraría notablemente los tiempos de respuesta a medida que la base de usuarios crezca.

Búsqueda avanzada con Elasticsearch

La búsqueda actual se apoya en consultas LIKE sobre MySQL. Migrar a Elasticsearch permitiría búsqueda full-text, sugerencias en tiempo real, filtros compuestos y búsqueda semántica.

Aplicación móvil nativa

Desarrollar una aplicación para iOS y Android con React Native, reutilizando la lógica de negocio ya presente en la API REST y los stores de Zustand. Las notificaciones push vía VAPID ya implementadas servirían de base.

Pipeline CI/CD

Incorporar GitHub Actions para ejecutar el build Maven, las pruebas y el despliegue automático en cada push a main. Actualmente el despliegue es manual mediante Docker Compose.

Batería de tests automatizados

El proyecto carece de tests unitarios e integración en el backend y de tests end-to-end en el frontend. Añadir JUnit 5 + Mockito en el backend y Playwright en el frontend aumentaría la confianza ante cambios futuros.

12.3 Mejoras de experiencia de usuario

Internacionalización (i18n)

Soporte multiidioma mediante react-i18next en el frontend y mensajes de error localizados en el backend, para abrir la plataforma a mercados más allá del hispanohablante.

Modo oscuro completo y temas

Aunque Tailwind CSS 4 facilita la implementación, el modo oscuro no está totalmente aplicado en todos los componentes. Completarlo y permitir al usuario elegir el tema mejoraría la accesibilidad visual.

Accesibilidad (WCAG 2.1 nivel AA)

Revisión de contraste de colores, navegación completa por teclado, atributos ARIA en componentes interactivos y etiquetas descriptivas en imágenes.

Panel de analíticas para administradores de comunidades

Métricas de actividad, crecimiento de miembros, publicaciones más reaccionadas y horas de mayor actividad, accesibles desde la pantalla de gestión de cada comunidad.

13. Conclusiones

13.1 Objetivos alcanzados

El proyecto Nexly ha cumplido en su totalidad los objetivos funcionales definidos al inicio del ciclo de desarrollo. Se ha construido una red social full-stack operativa con las siguientes áreas completamente implementadas:

Autenticación completa: registro con verificación de email por token, inicio de sesión con JWT almacenado en cookie HttpOnly, e inicio de sesión federado con Google OAuth2. El flujo de seguridad cubre renovación de tokens, cierre de sesión y protección de rutas tanto en el frontend como en el backend.

Publicaciones y feed: feed paginado con cinco tipos de publicación (normal, pregunta, noticia, debate y anuncio), soporte de imágenes múltiples via Cloudinary, publicaciones citadas con componente QuoteCard, reacciones, publicaciones fijadas en comunidades y paginación por cursor.

Comentarios anidados: sistema de comentarios con un nivel de respuesta, cada uno con su propio contador de reacciones, implementado tanto en el modelo de datos JPA como en el componente React correspondiente.

Perfiles y relaciones sociales: edición de perfil con avatar, seguimiento con soporte de perfiles privados (solicitud pendiente), bloqueo de usuarios y visibilidad diferenciada según el estado de la relación. Comunidades: creación con categoría e imagen, sistema de roles jerárquico (ADMIN, MODERADOR, MIEMBRO), solicitudes de ingreso en comunidades privadas, expulsión de miembros y moderación de contenido.

Mensajería en tiempo real: chat privado y de grupo con WebSocket STOMP/SockJS, marcado de mensajes como leídos y estado de conversación persistido en base de datos. Notificaciones: notificaciones in-app en tiempo real via WebSocket y notificaciones push para dispositivos mediante Web Push con claves VAPID, con soporte de PWA.

Panel de administración: dashboard con estadísticas globales, gestión de usuarios (activar/desactivar, cambiar rol), moderación de publicaciones y comunidades, con paginación propia.

Infraestructura reproducible: entorno de desarrollo completamente contenedorizado con Docker Compose, incluyendo base de datos MySQL, servidor de correo Mailpit y el propio backend Spring Boot, acompañado de un script data.sql idempotente con datos de prueba.

13.2. Objetivos no alcanzados y causas

Durante el desarrollo fue necesario descartar la funcionalidad de **encuestas en publicaciones**, que estaba prevista en el diseño inicial.

La decisión se tomó al constatar que su implementación completa —modelo de datos, API, componente de votación en tiempo real y visualización de resultados— requería más tiempo del disponible en la última fase del proyecto, coincidiendo con la integración del sistema de notificaciones push y la finalización del panel de administración.

Se priorizaron estas últimas por considerarse más representativas de las competencias técnicas evaluables del ciclo. Asimismo, la batería de tests automatizados no llegó a implementarse.

En un contexto profesional, las pruebas unitarias e integración serían obligatorias antes del despliegue; en el marco del proyecto de fin de ciclo, el tiempo se destinó a ampliar el número de funcionalidades implementadas y a garantizar su correcto funcionamiento de forma manual.

13.3. Valoración personal

El desarrollo de Nexly ha supuesto la integración práctica de la mayor parte de los contenidos trabajados a lo largo del ciclo: diseño y normalización de bases de datos relacionales, construcción de una API REST segura con Spring Boot y Spring Security, desarrollo de una SPA moderna con React y TypeScript, comunicación asíncrona y en tiempo real con WebSockets, y despliegue con contenedores Docker.

Desde el punto de vista técnico, los aspectos más enriquecedores han sido la implementación del flujo OAuth2 con Google, la gestión del estado en tiempo real con STOMP y Zustand, y la construcción del sistema de notificaciones push con VAPID, tecnologías que van más allá del currículo habitual del módulo y que han requerido consultar documentación oficial y especificaciones de la W3C.

El resultado es una plataforma funcional, con una arquitectura limpia y escalable, lista para ser desplegada en producción con mínimas modificaciones, y que demuestra la capacidad de abordar un proyecto de software complejo de principio a fin de forma autónoma.

14. Biblioteca de recursos web y referencias

Apache Software Foundation (2024): Maven Getting Started Guide, Apache Software Foundation, disponible en:

<https://maven.apache.org/guides/getting-started/>

Banks, A. y Porcello, E. (2020): Learning React: Modern Patterns for Developing React Apps, 2.^a ed., O'Reilly Media, pp. 1–312.

Bauer, C., King, G. y Gregory, G. (2015): Java Persistence with Hibernate, 2.^a ed., Manning Publications, pp. 1–600.

Cherny, B. (2019): Programming TypeScript: Making Your JavaScript Applications Scale, O'Reilly Media, pp. 1–330.

Cloudinary Ltd. (2024): Cloudinary Developer Documentation — Java SDK & Upload API, Cloudinary Ltd., disponible en:

https://cloudinary.com/documentation/java_integration

Docker Inc. (2024): Docker Documentation — Get Started, Docker Inc., disponible en: <https://docs.docker.com/get-started/>

Google LLC (2024): Using OAuth 2.0 to Access Google APIs, Google Developers, disponible en: <https://developers.google.com/identity/protocols/oauth2>

Hardt, D. (Ed.) (2012): RFC 6749: The OAuth 2.0 Authorization Framework, IETF, disponible en: <https://datatracker.ietf.org/doc/html/rfc6749>

Jones, M., Bradley, J. y Sakimura, N. (2015): RFC 7519: JSON Web Token (JWT), IETF, disponible en: <https://datatracker.ietf.org/doc/html/rfc7519>

Kato, D. (2023): Micro State Management with React Hooks, Packt Publishing, pp. 96–120.

Meta Open Source (2024): React — The Library for Web and Native User Interfaces, Meta Open Source, disponible en: <https://react.dev>

Microsoft Corporation (2024): TypeScript Handbook, Microsoft Corporation, disponible en: <https://www.typescriptlang.org/docs/handbook/intro.html>

Mozilla Foundation (2024a): Push API — MDN Web Docs, Mozilla Foundation, disponible en: https://developer.mozilla.org/en-US/docs/Web/API/Push_API

Mozilla Foundation (2024b): Service Worker API — MDN Web Docs, Mozilla Foundation, disponible en: https://developer.mozilla.org/en-US/docs/Web/API/Service_Worker_API

Mozilla Foundation (2024c): WebSocket API — MDN Web Docs, Mozilla Foundation, disponible en: <https://developer.mozilla.org/en-US/docs/Web/API/WebSocket>

MySQL AB / Oracle Corporation (2024): MySQL 8.0 Reference Manual, Oracle Corporation, disponible en: <https://dev.mysql.com/doc/refman/8.0/en/>

Pivotal / VMware (2024): Spring Boot Reference Documentation, Spring Framework Team, disponible en: <https://docs.spring.io/spring-boot/docs/current/reference/html/>

Poulton, N. (2023): Docker Deep Dive, Independently Published, pp. 1–220, disponible en: <https://nigelpoulton.com/books/>

Remix Software Inc. (2024): React Router v7 — Framework Documentation, Remix Software Inc., disponible en: <https://reactrouter.com/en/main>